

main

November 29, 2025

```
[364]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

1 Import Data

```
[365]: df=pd.read_csv('heart.csv')
```

```
[366]: df.shape
```

```
[366]: (918, 12)
```

```
[367]: df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 918 entries, 0 to 917
Data columns (total 12 columns):
#   Column          Non-Null Count  Dtype
---  -
0   Age             918 non-null   int64
1   Sex             918 non-null   object
2   ChestPainType   918 non-null   object
3   RestingBP       918 non-null   int64
4   Cholesterol     918 non-null   int64
5   FastingBS       918 non-null   int64
6   RestingECG      918 non-null   object
7   MaxHR           918 non-null   int64
8   ExerciseAngina  918 non-null   object
9   Oldpeak         918 non-null   float64
10  ST_Slope        918 non-null   object
11  HeartDisease    918 non-null   int64
dtypes: float64(1), int64(6), object(5)
memory usage: 86.2+ KB
```

```
[368]: df.describe()
```

```
[368]:
```

	Age	RestingBP	Cholesterol	FastingBS	MaxHR	\
count	918.000000	918.000000	918.000000	918.000000	918.000000	
mean	53.510893	132.396514	198.799564	0.233115	136.809368	
std	9.432617	18.514154	109.384145	0.423046	25.460334	
min	28.000000	0.000000	0.000000	0.000000	60.000000	
25%	47.000000	120.000000	173.250000	0.000000	120.000000	
50%	54.000000	130.000000	223.000000	0.000000	138.000000	
75%	60.000000	140.000000	267.000000	0.000000	156.000000	
max	77.000000	200.000000	603.000000	1.000000	202.000000	

	Oldpeak	HeartDisease
count	918.000000	918.000000
mean	0.887364	0.553377
std	1.066570	0.497414
min	-2.600000	0.000000
25%	0.000000	0.000000
50%	0.600000	1.000000
75%	1.500000	1.000000
max	6.200000	1.000000

2 Handle missing values

```
[369]: df.isnull().sum()
```

```
[369]: Age          0
Sex              0
ChestPainType    0
RestingBP        0
Cholesterol       0
FastingBS        0
RestingECG       0
MaxHR            0
ExerciseAngina   0
Oldpeak          0
ST_Slope         0
HeartDisease     0
dtype: int64
```

3 Handling duplicates

```
[370]: df.duplicated().sum()
```

```
[370]: 0
```

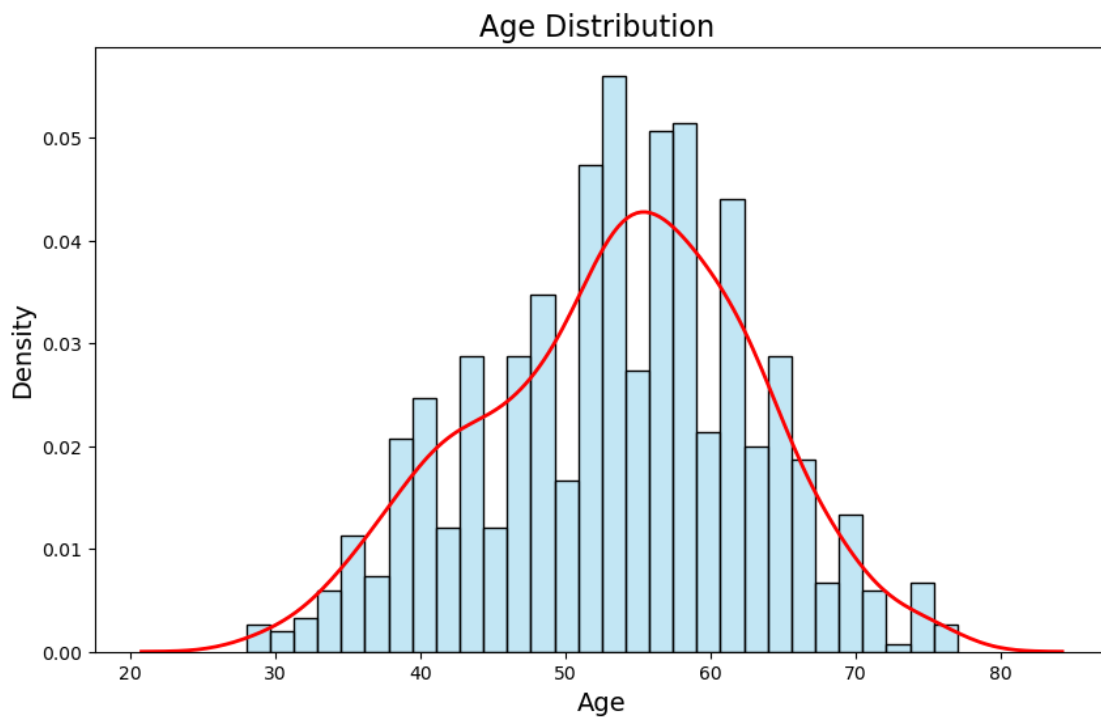
4 Exploratory Data Analysis , Data Cleaning & Data Preprocessing

4.1 Single Variable

4.1.1 Age

```
[371]: plt.figure(figsize=(10, 6))
sns.histplot(df['Age'], bins=30, kde=True, color='skyblue', stat='density',
             alpha=0.5)
sns.kdeplot(df['Age'], color='red', linewidth=2)
plt.title('Age Distribution ', fontsize=16)
plt.xlabel('Age', fontsize=14)
plt.ylabel('Density', fontsize=14)

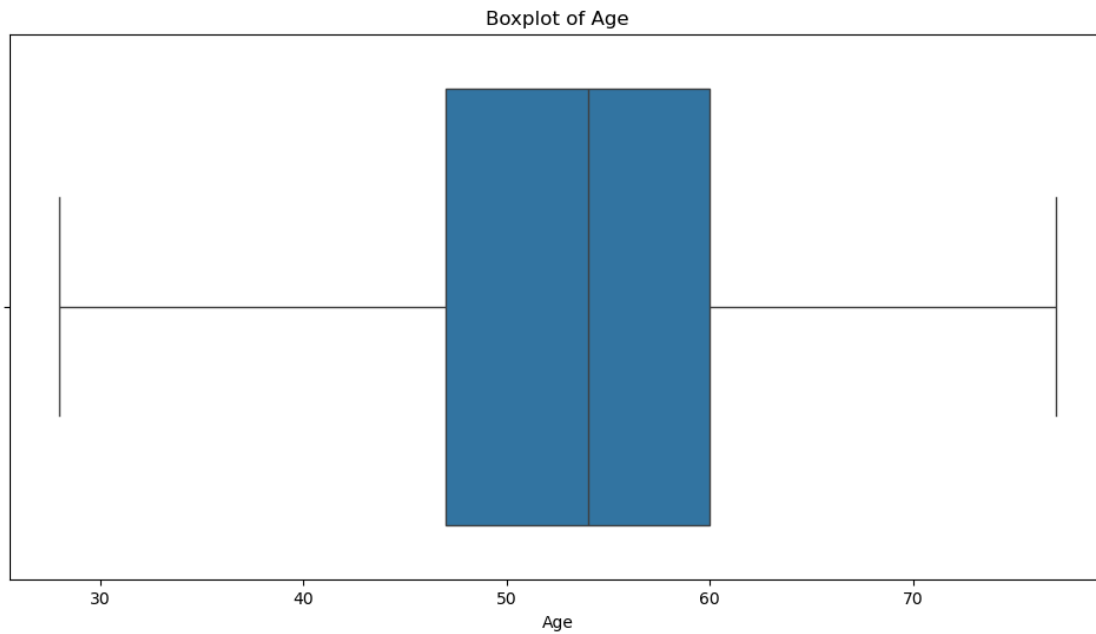
# Show the plot
plt.show()
print(df.Age.describe())
```



```
count    918.000000
mean     53.510893
std       9.432617
min      28.000000
25%      47.000000
50%      54.000000
```

```
75%      60.000000
max      77.000000
Name: Age, dtype: float64
```

```
[372]: plt.figure(figsize=(12, 6))
sns.boxplot(x='Age', data=df)
plt.title('Boxplot of Age')
plt.xlabel('Age')
plt.show()
```

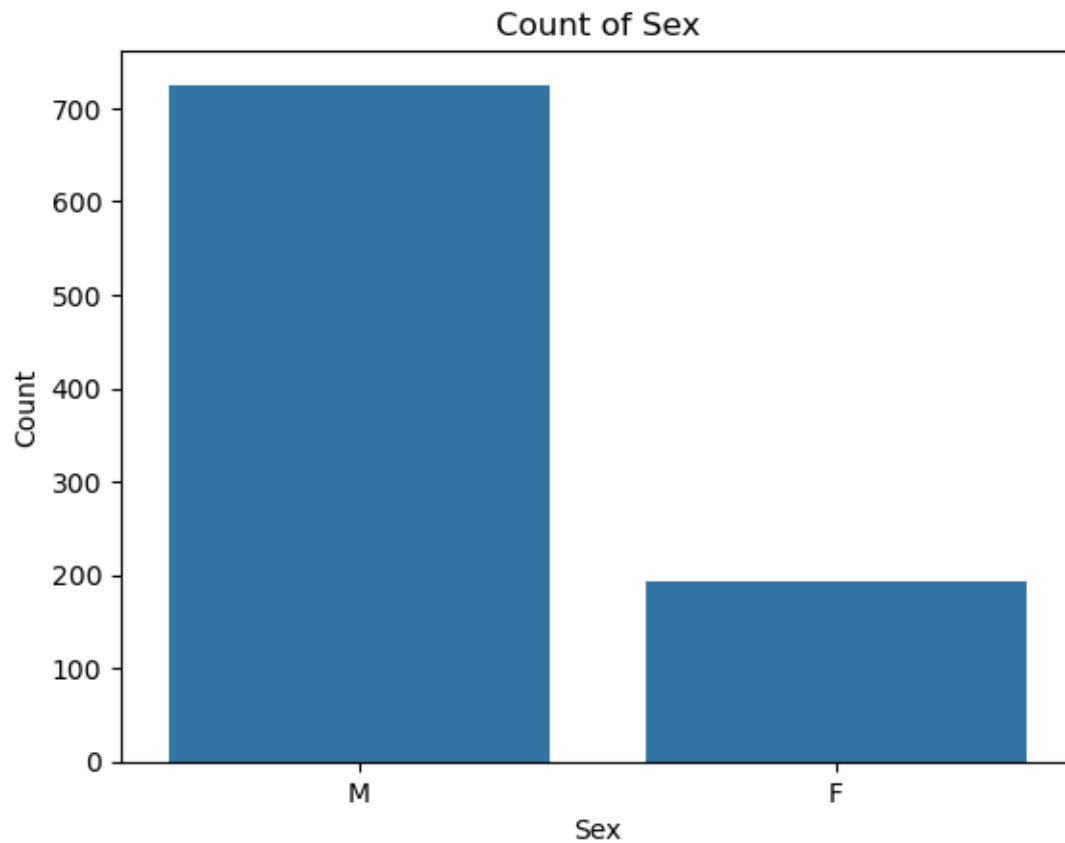


- no noisy data or outliers found

4.1.2 Sex

```
[373]: sns.countplot(x='Sex', data=df)
plt.title('Count of Sex')
plt.xlabel('Sex')
plt.ylabel('Count')
plt.show()

df['Sex'].value_counts()
```

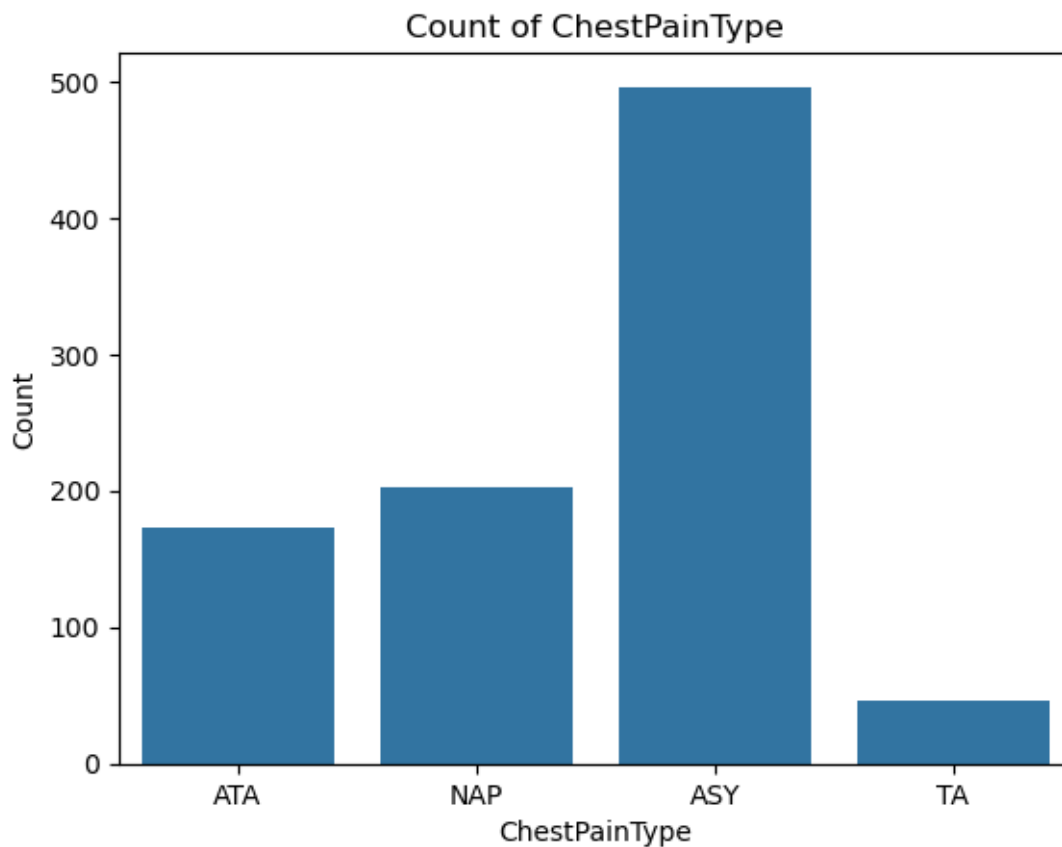


```
[373]: Sex
      M    725
      F    193
      Name: count, dtype: int64
```

4.1.3 ChestPainType

```
[374]: sns.countplot(x='ChestPainType', data=df)
plt.title('Count of ChestPainType')
plt.xlabel('ChestPainType')
plt.ylabel('Count')
plt.show()

df['ChestPainType'].value_counts()
```



[374]: ChestPainType

```
ASY    496
NAP    203
ATA    173
TA      46
Name: count, dtype: int64
```

[375]: `df['ChestPainType'].value_counts()/df.shape[0] *100 # ChestPainType_`
↪distribution percentage

[375]: ChestPainType

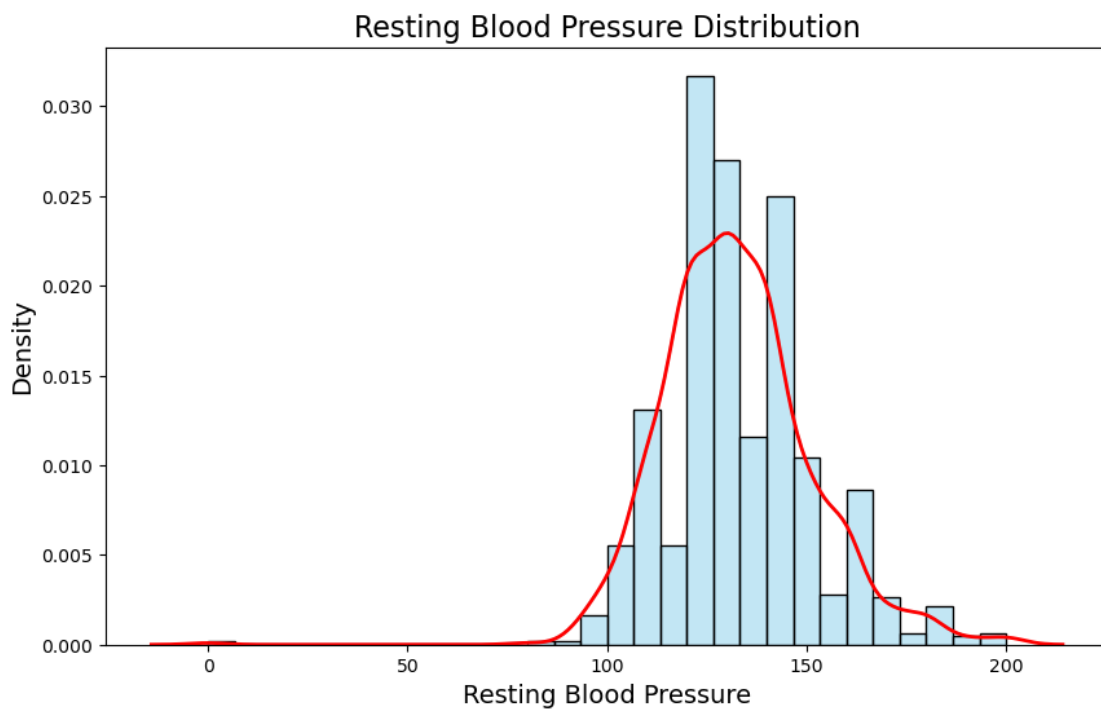
```
ASY    54.030501
NAP    22.113290
ATA    18.845316
TA      5.010893
Name: count, dtype: float64
```

- The data shows the distribution of chest pain types, with ASY being the most common (54.03%), followed by NAP (22.11%), ATA (18.85%), and TA (5.01%).

4.1.4 RestingBP

```
[376]: plt.figure(figsize=(10, 6))
sns.histplot(df['RestingBP'], bins=30, kde=True, color='skyblue',
             stat='density', alpha=0.5)
sns.kdeplot(df['RestingBP'], color='red', linewidth=2)
plt.title('Resting Blood Pressure Distribution', fontsize=16)
plt.xlabel('Resting Blood Pressure', fontsize=14)
plt.ylabel('Density', fontsize=14)

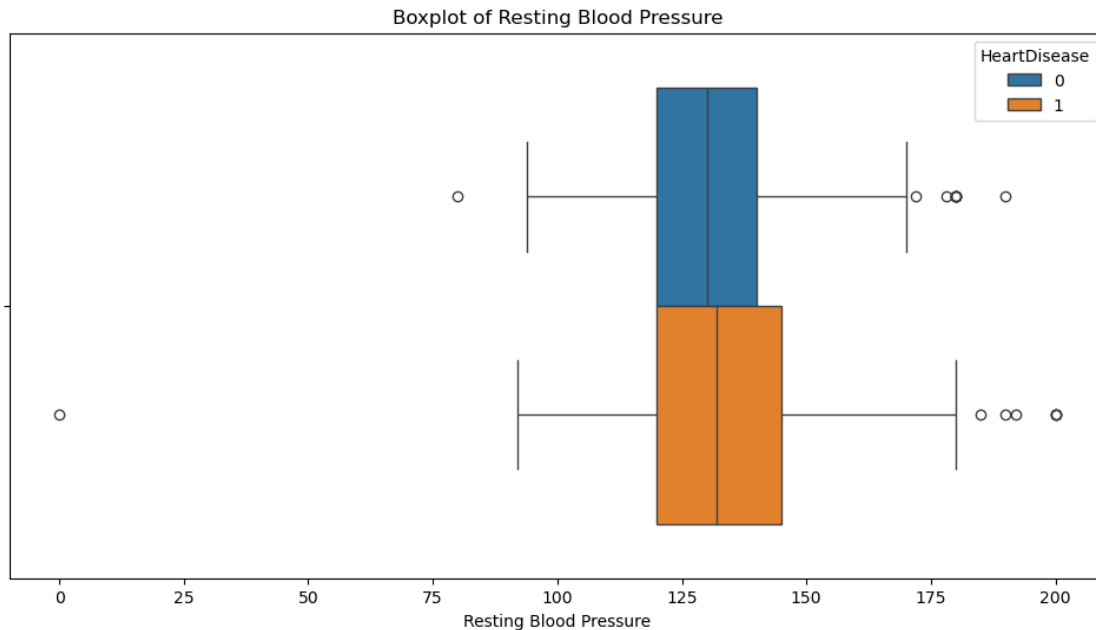
# Show the plot
plt.show()
print(df.RestingBP.describe())
```



```
count    918.000000
mean     132.396514
std       18.514154
min        0.000000
25%      120.000000
50%      130.000000
75%      140.000000
max       200.000000
Name: RestingBP, dtype: float64
```

- bell-shaped distribution

```
[377]: plt.figure(figsize=(12, 6))
sns.boxplot(x='RestingBP', data=df, hue='HeartDisease')
plt.title('Boxplot of Resting Blood Pressure')
plt.xlabel('Resting Blood Pressure')
plt.show()
```



```
[378]: Q1 = df['RestingBP'].quantile(0.25)
Q3 = df['RestingBP'].quantile(0.75)
max = df['RestingBP'].max()
min = df['RestingBP'].min()
print(f"Q1: {Q1}, Q3: {Q3}, Max: {max}, Min: {min}")
IQR = Q3 - Q1

lower_bound = Q1 - 1.5 * IQR
upper_bound = Q3 + 1.5 * IQR

outliers_normal = df[(df['RestingBP'] < lower_bound) | (df['RestingBP'] >
    ↳ upper_bound) ] & (df['HeartDisease'] == 0)]
outliers_heartDisease = df[(df['RestingBP'] < lower_bound) | (df['RestingBP']
    ↳ > upper_bound) ] & (df['HeartDisease'] == 1)]
# Count the number of outliers
print(f'Number of outliers (normal): {outliers_normal.shape[0]}')
print(f'Number of outliers (heart disease): {outliers_heartDisease.shape[0]}')
```

```
Q1: 120.0, Q3: 140.0, Max: 200, Min: 0
Number of outliers (normal): 10
```


Number of outliers (heart disease): 18

- we have 28 outliers in total for this attribute

```
[379]: # Display the outliers
df[(df['RestingBP'] < lower_bound) | (df['RestingBP'] > upper_bound)].loc[:,
↳ ['RestingBP', 'HeartDisease']]
```

```
[379]:
```

	RestingBP	HeartDisease
109	190	0
123	180	1
189	180	1
190	180	0
241	200	1
274	180	0
275	180	0
278	180	0
314	80	0
365	200	1
372	185	1
399	200	1
411	180	1
423	180	0
449	0	1
475	178	1
550	172	1
585	180	1
592	190	1
673	174	1
702	178	0
725	180	1
732	200	1
759	192	1
774	178	1
780	180	0
855	180	1
880	172	0

Analyze whether this would have an effect on our response variable - Heart Disease

Handle Noise

- 0 Resting BP must be an invalid data due to error in collection of data, we will remove this row
- In medical context, resting BP around 180 is treated as (severe hypertension), it is still valid data point, we will keep it for analysis as 27 outliers is still moderate amount in 918 observations

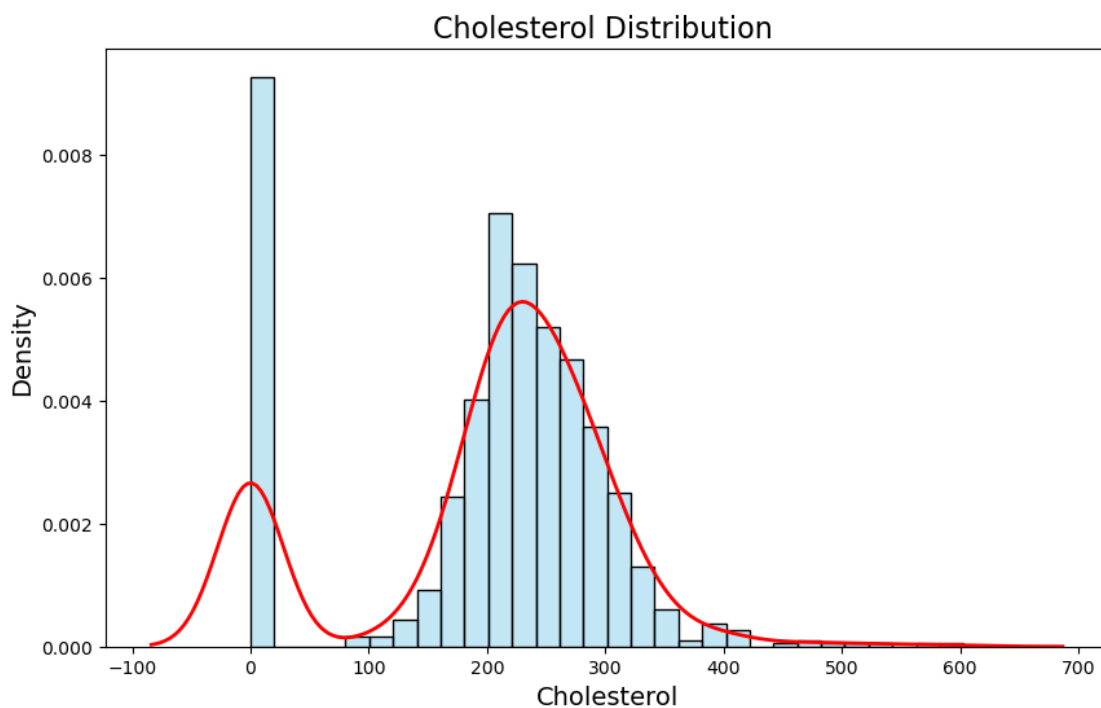
```
[380]: df=df[df['RestingBP'] != 0]
df.shape
```

```
[380]: (917, 12)
```

4.1.5 Cholesterol

```
[381]: plt.figure(figsize=(10, 6))
sns.histplot(df['Cholesterol'], bins=30, kde=True, color='skyblue',
             stat='density', alpha=0.5)
sns.kdeplot(df['Cholesterol'], color='red', linewidth=2)
plt.title('Cholesterol Distribution ', fontsize=16)
plt.xlabel('Cholesterol', fontsize=14)
plt.ylabel('Density', fontsize=14)

# Show the plot
plt.show()
print(df.Cholesterol.describe())
```



```
count    917.000000
mean     199.016358
std      109.246330
min        0.000000
25%      174.000000
50%      223.000000
```

```
75%      267.000000
max      603.000000
Name: Cholesterol, dtype: float64
```

- bimodal distribution identified, with a significant amount of data cholesterol = 0

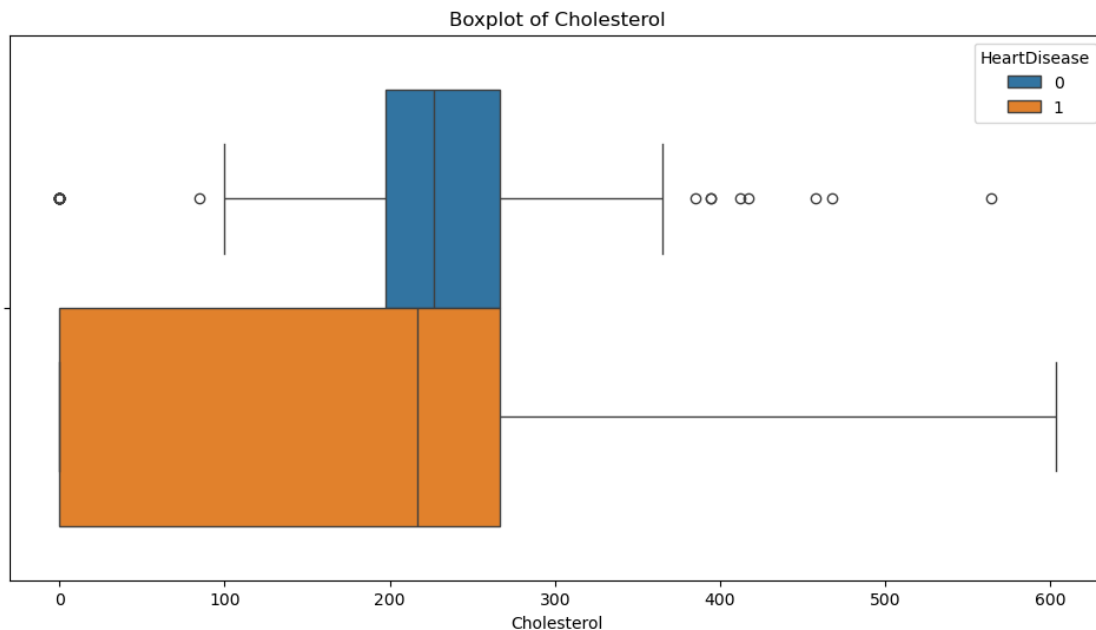
Handle Noise

```
[382]: df[df['Cholesterol'] == 0].count()
```

```
[382]: Age      171
      Sex      171
      ChestPainType  171
      RestingBP    171
      Cholesterol  171
      FastingBS    171
      RestingECG   171
      MaxHR       171
      ExerciseAngina  171
      Oldpeak      171
      ST_Slope     171
      HeartDisease  171
      dtype: int64
```

- 171 cholesterol values are 0

```
[383]: plt.figure(figsize=(12, 6))
      sns.boxplot(x='Cholesterol', data=df, hue='HeartDisease')
      plt.title('Boxplot of Cholesterol')
      plt.xlabel('Cholesterol')
      plt.show()
```



- Since 0 cholesterol level is impossible in medical context, these zero values are interpreted as miscollect data or values that are imputed manually
- We would remove all these 171 values

```
[384]: df=df[df['Cholesterol']!=0]
```

```
[385]: df
```

```
[385]:
```

	Age	Sex	ChestPainType	RestingBP	Cholesterol	FastingBS	RestingECG	\
0	40	M	ATA	140	289	0	Normal	
1	49	F	NAP	160	180	0	Normal	
2	37	M	ATA	130	283	0	ST	
3	48	F	ASY	138	214	0	Normal	
4	54	M	NAP	150	195	0	Normal	
..	...	..	...	...	...	...	...	
913	45	M	TA	110	264	0	Normal	
914	68	M	ASY	144	193	1	Normal	
915	57	M	ASY	130	131	0	Normal	
916	57	F	ATA	130	236	0	LVH	
917	38	M	NAP	138	175	0	Normal	

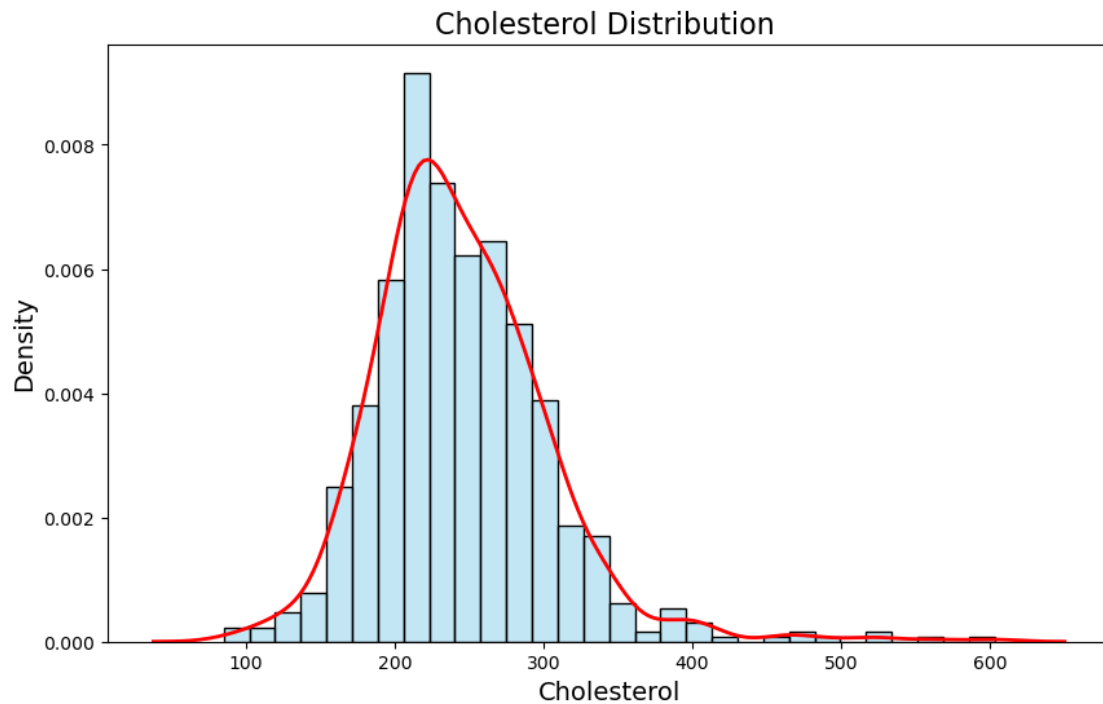
	MaxHR	ExerciseAngina	Oldpeak	ST_Slope	HeartDisease
0	172	N	0.0	Up	0
1	156	N	1.0	Flat	1
2	98	N	0.0	Up	0
3	108	Y	1.5	Flat	1
4	122	N	0.0	Up	0
..	...	...	...	...	...
913	132	N	1.2	Flat	1
914	141	N	3.4	Flat	1
915	115	Y	1.2	Flat	1
916	174	N	0.0	Flat	1
917	173	N	0.0	Up	0

[746 rows x 12 columns]

- Re-plot Histogram & box plot again

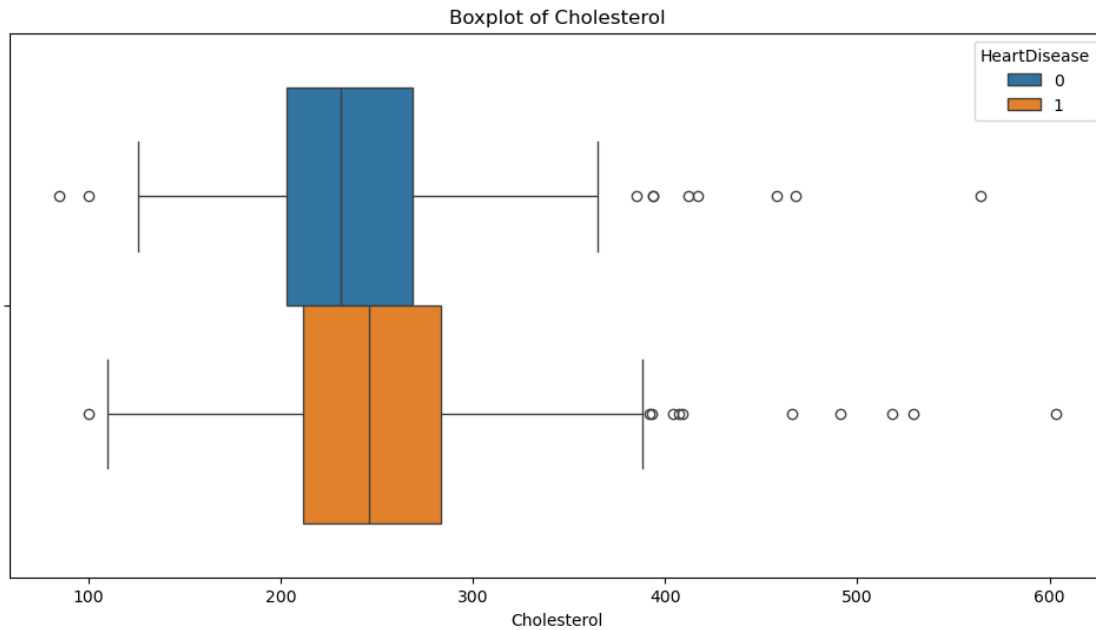
```
[386]: plt.figure(figsize=(10, 6))
sns.histplot(df['Cholesterol'], bins=30, kde=True, color='skyblue',
stat='density', alpha=0.5)
sns.kdeplot(df['Cholesterol'], color='red', linewidth=2)
plt.title('Cholesterol Distribution ', fontsize=16)
plt.xlabel('Cholesterol', fontsize=14)
plt.ylabel('Density', fontsize=14)
```

```
# Show the plot
plt.show()
print(df.Cholesterol.describe())
```



```
count    746.000000
mean     244.635389
std       59.153524
min       85.000000
25%      207.250000
50%      237.000000
75%      275.000000
max       603.000000
Name: Cholesterol, dtype: float64
```

```
[387]: plt.figure(figsize=(12, 6))
sns.boxplot(x='Cholesterol', data=df, hue='HeartDisease')
plt.title('Boxplot of Cholesterol')
plt.xlabel('Cholesterol')
plt.show()
```



```
[388]: Q1 = df['Cholesterol'].quantile(0.25)
Q3 = df['Cholesterol'].quantile(0.75)
max = df['Cholesterol'].max()
min = df['Cholesterol'].min()
print(f"Q1: {Q1}, Q3: {Q3}, Max: {max}, Min: {min}")
IQR = Q3 - Q1

lower_bound = Q1 - 1.5 * IQR
upper_bound = Q3 + 1.5 * IQR

outliers_normal = df[(df['Cholesterol'] < lower_bound) | (df['Cholesterol'] >
    ↳ upper_bound) ] & (df['HeartDisease'] == 0)]
outliers_heartDisease = df[(df['Cholesterol'] < lower_bound) |
    ↳ (df['Cholesterol'] > upper_bound) ] & (df['HeartDisease'] == 1)]
# Count the number of outliers
print(f'Number of outliers (normal): {outliers_normal.shape[0]}')
print(f'Number of outliers (heart disease): {outliers_heartDisease.shape[0]}')
```

Q1: 207.25, Q3: 275.0, Max: 603, Min: 85

Number of outliers (normal): 10

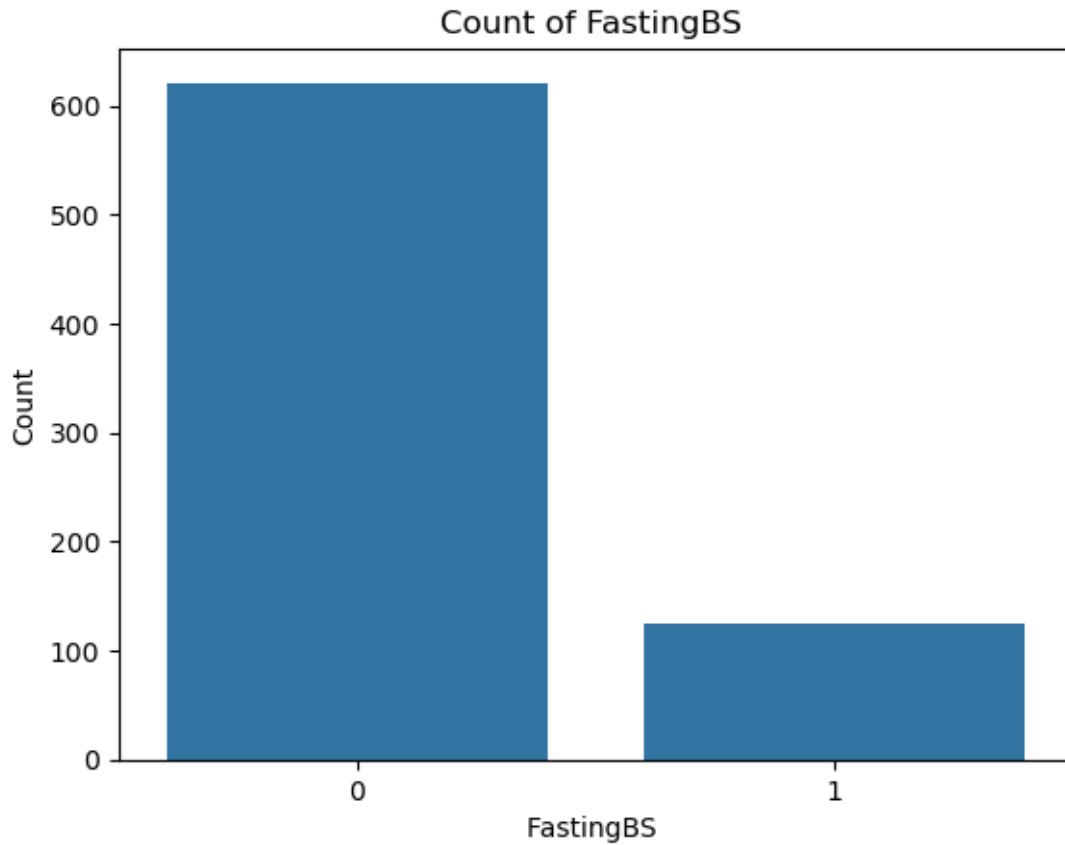
Number of outliers (heart disease): 13

- both class have similar number of outliers and 23 outliers in 746 observations (around 3 %) is moderate. We would keep the remaining in our analysis

4.1.6 FastingBS

```
[389]: sns.countplot(x='FastingBS', data=df)
plt.title('Count of FastingBS ')
plt.xlabel('FastingBS ')
plt.ylabel('Count')
plt.show()

df['FastingBS'].value_counts()
```

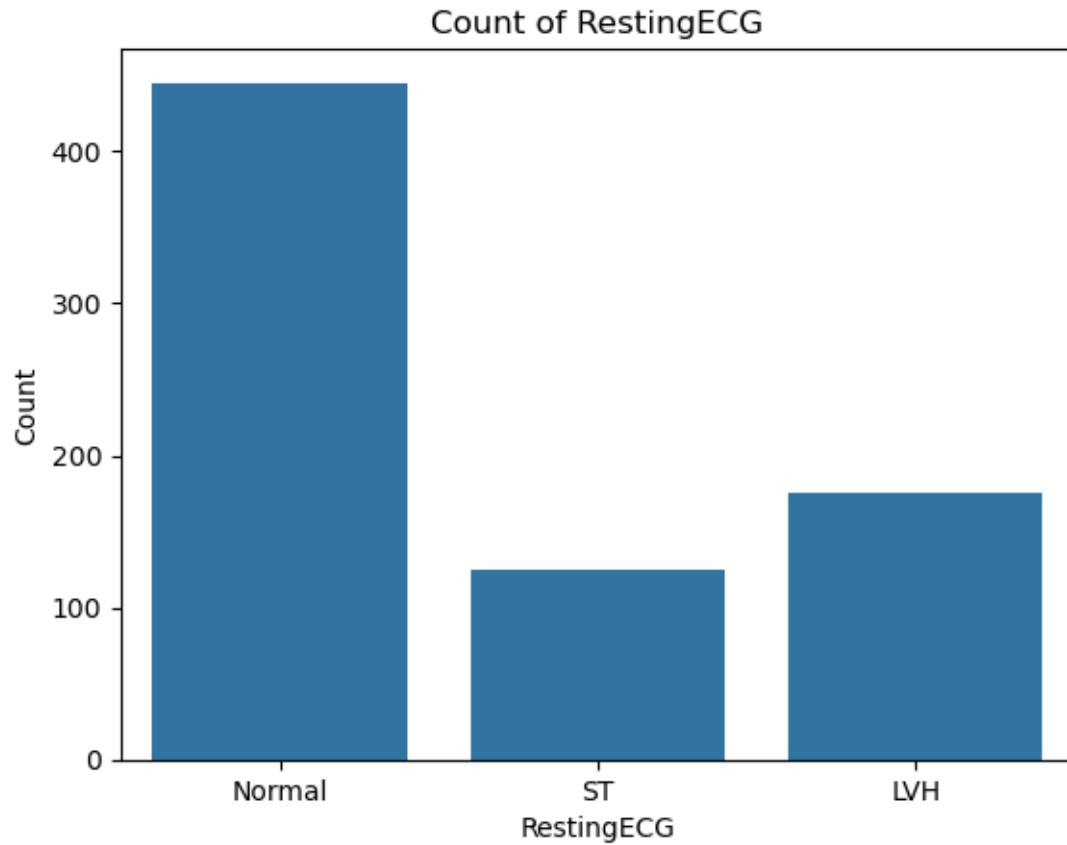


```
[389]: FastingBS
0      621
1      125
Name: count, dtype: int64
```

4.1.7 RestingECG

```
[390]: sns.countplot(x='RestingECG', data=df)
plt.title('Count of RestingECG')
plt.xlabel('RestingECG')
plt.ylabel('Count')
plt.show()

df['RestingECG'].value_counts()
```

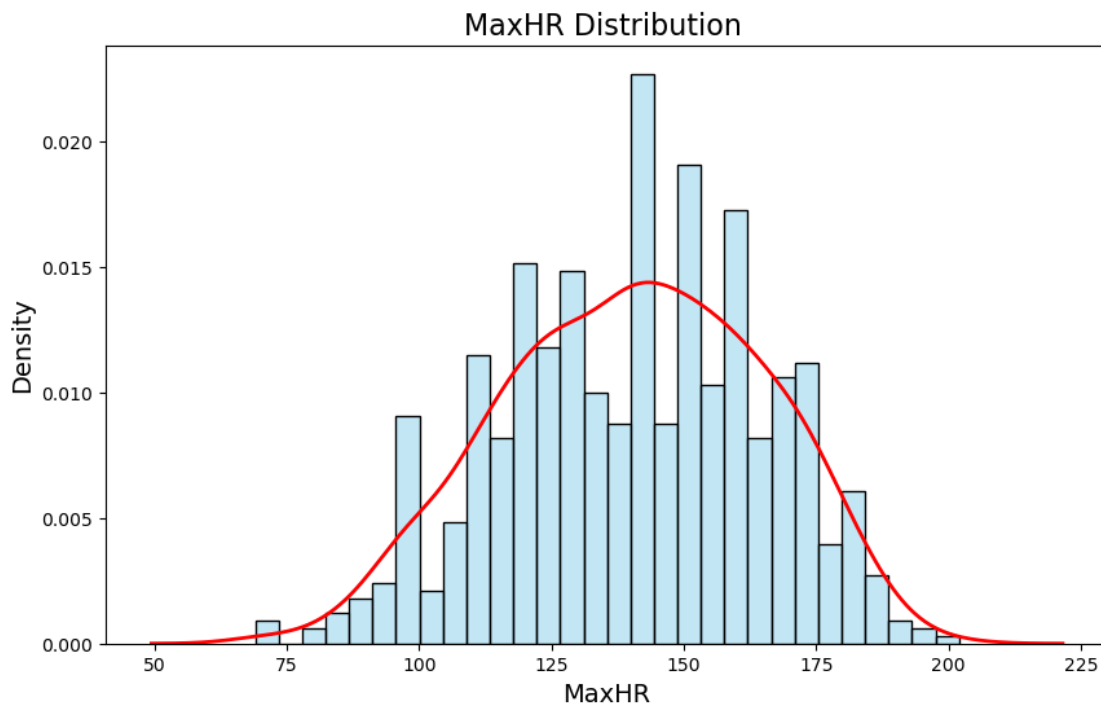


```
[390]: RestingECG
Normal    445
LVH       176
ST        125
Name: count, dtype: int64
```


4.1.8 MaxHR

```
[391]: plt.figure(figsize=(10, 6))
sns.histplot(df['MaxHR'], bins=30, kde=True, color='skyblue', stat='density',
             alpha=0.5)
sns.kdeplot(df['MaxHR'], color='red', linewidth=2)
plt.title('MaxHR Distribution ', fontsize=16)
plt.xlabel('MaxHR', fontsize=14)
plt.ylabel('Density', fontsize=14)

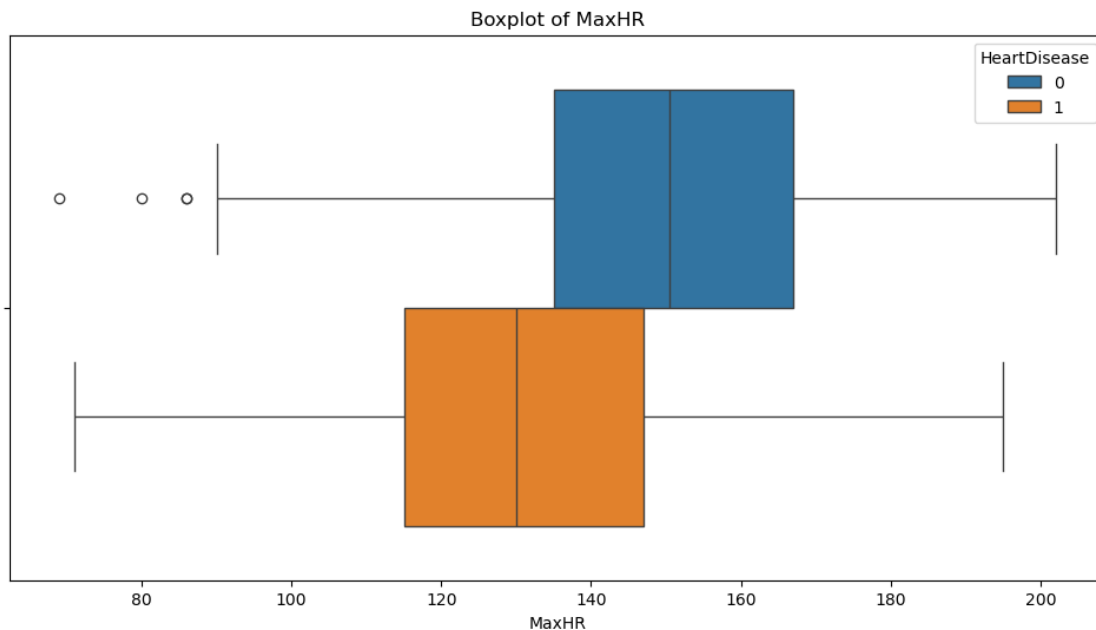
# Show the plot
plt.show()
print(df.MaxHR.describe())
```



```
count      746.000000
mean       140.226542
std        24.524107
min         69.000000
25%        122.000000
50%        140.000000
75%        160.000000
max        202.000000
Name: MaxHR, dtype: float64
```

- bell-shaped distribution

```
[392]: plt.figure(figsize=(12, 6))
sns.boxplot(x='MaxHR', data=df, hue='HeartDisease')
plt.title('Boxplot of MaxHR')
plt.xlabel('MaxHR')
plt.show()
```

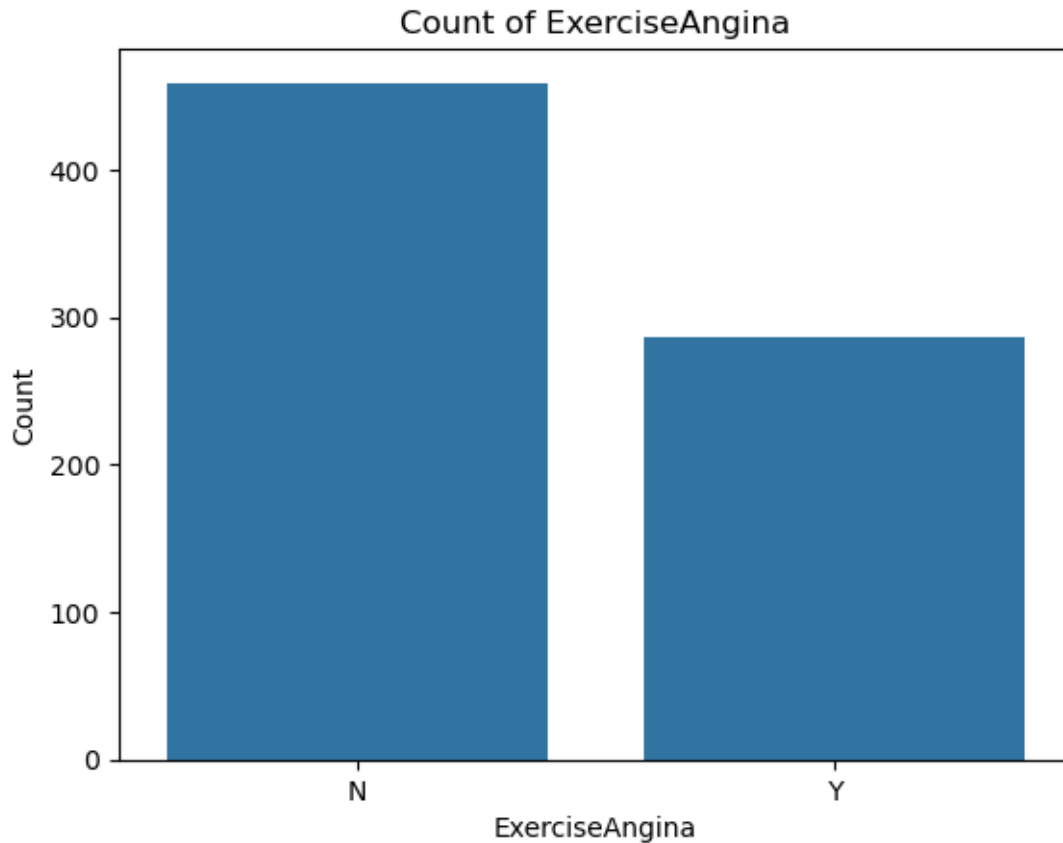


- just few number of outliers and all fall in reasonable range, we will keep all of them

4.1.9 ExerciseAngina:

```
[393]: sns.countplot(x='ExerciseAngina', data=df)
plt.title('Count of ExerciseAngina')
plt.xlabel('ExerciseAngina')
plt.ylabel('Count')
plt.show()

df['ExerciseAngina'].value_counts()
```

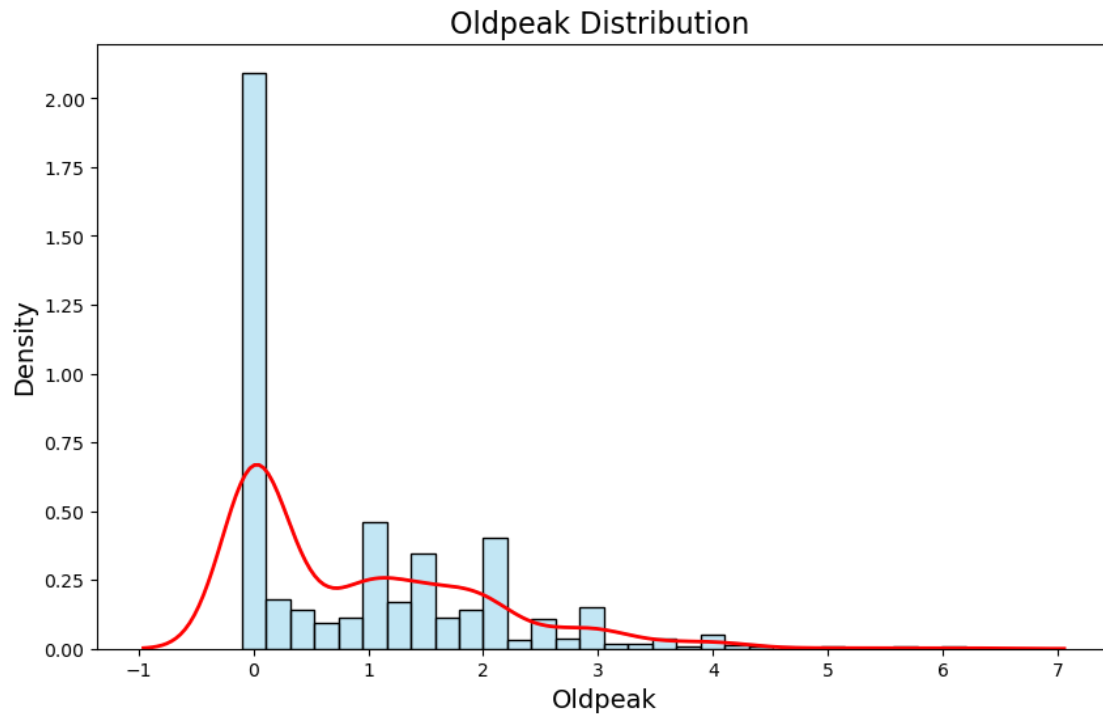


```
[393]: ExerciseAngina
      N    459
      Y    287
      Name: count, dtype: int64
```

4.1.10 Oldpeak

```
[394]: plt.figure(figsize=(10, 6))
sns.histplot(df['Oldpeak'], bins=30, kde=True, color='skyblue', stat='density',
             alpha=0.5)
sns.kdeplot(df['Oldpeak'], color='red', linewidth=2)
plt.title('Oldpeak Distribution ', fontsize=16)
plt.xlabel('Oldpeak', fontsize=14)
plt.ylabel('Density', fontsize=14)

# Show the plot
plt.show()
print(df.Oldpeak.describe())
```



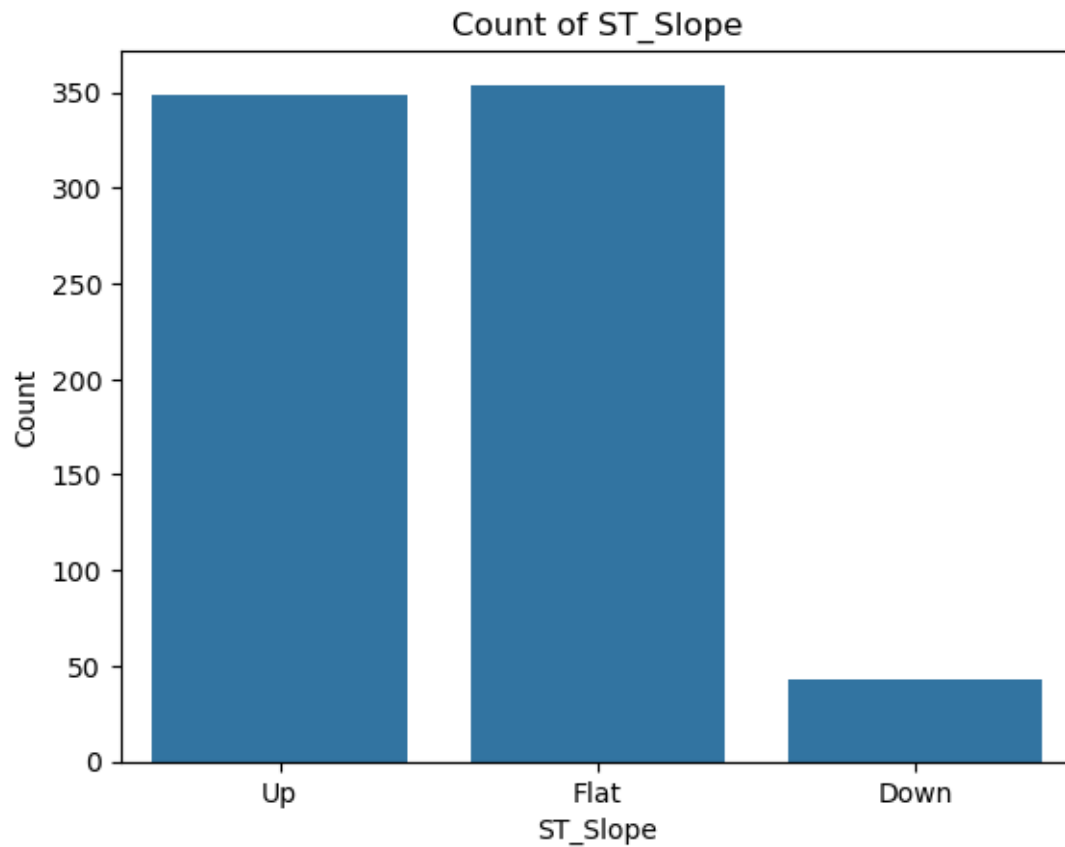
```
count    746.000000
mean      0.901609
std       1.072861
min       -0.100000
25%       0.000000
50%       0.500000
75%       1.500000
max        6.200000
Name: Oldpeak, dtype: float64
```

- skewed distribution identified; standardization might be needed

4.1.11 ST_Slope

```
[395]: sns.countplot(x='ST_Slope', data=df)
plt.title('Count of ST_Slope')
plt.xlabel('ST_Slope')
plt.ylabel('Count')
plt.show()

df['ST_Slope'].value_counts()
```

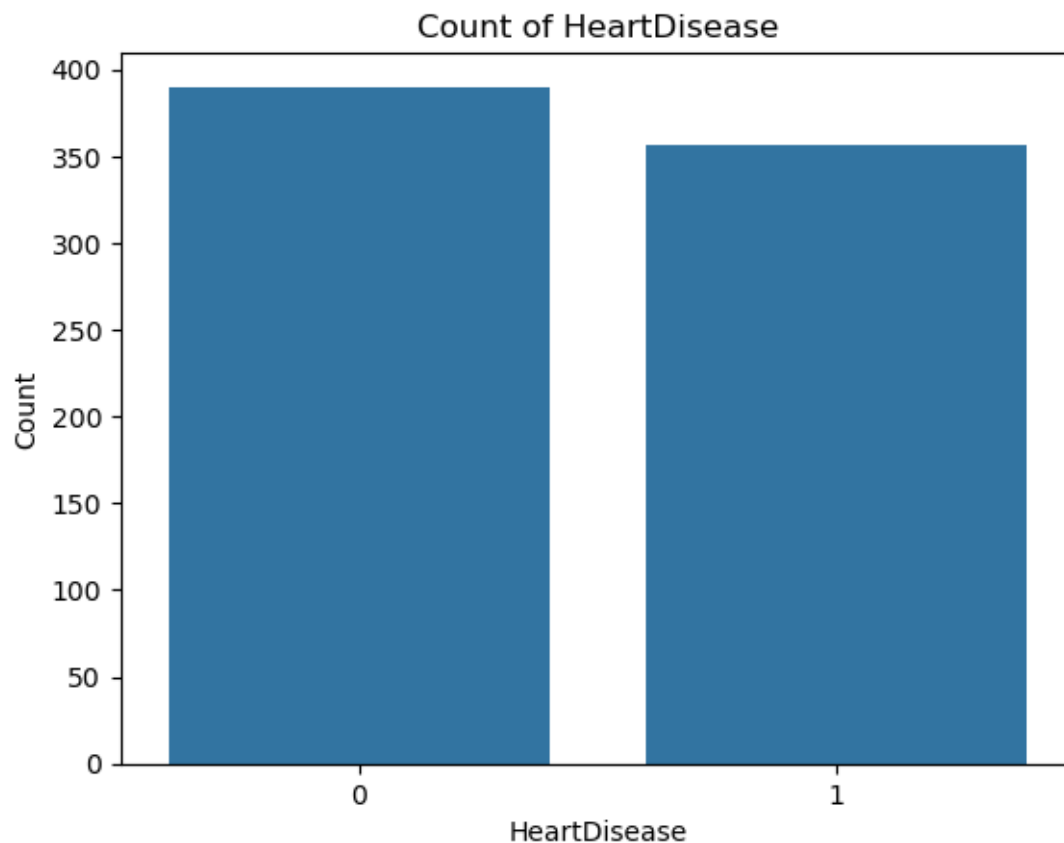


```
[395]: ST_Slope
      Flat    354
      Up     349
      Down    43
      Name: count, dtype: int64
```

4.1.12 Y : HeartDisease

```
[396]: sns.countplot(x='HeartDisease', data=df)
      plt.title('Count of HeartDisease')
      plt.xlabel('HeartDisease')
      plt.ylabel('Count')
      plt.show()

      df['HeartDisease'].value_counts()
```



```
[396]: HeartDisease
```

```
0    390
```

```
1    356
```

```
Name: count, dtype: int64
```

- classes of response are almost balanced

```
[397]: df.to_csv('heart-cleaned.csv', index=False)
```

4.2 Correlation analysis

```
[398]: df.corr(numeric_only=True)
```

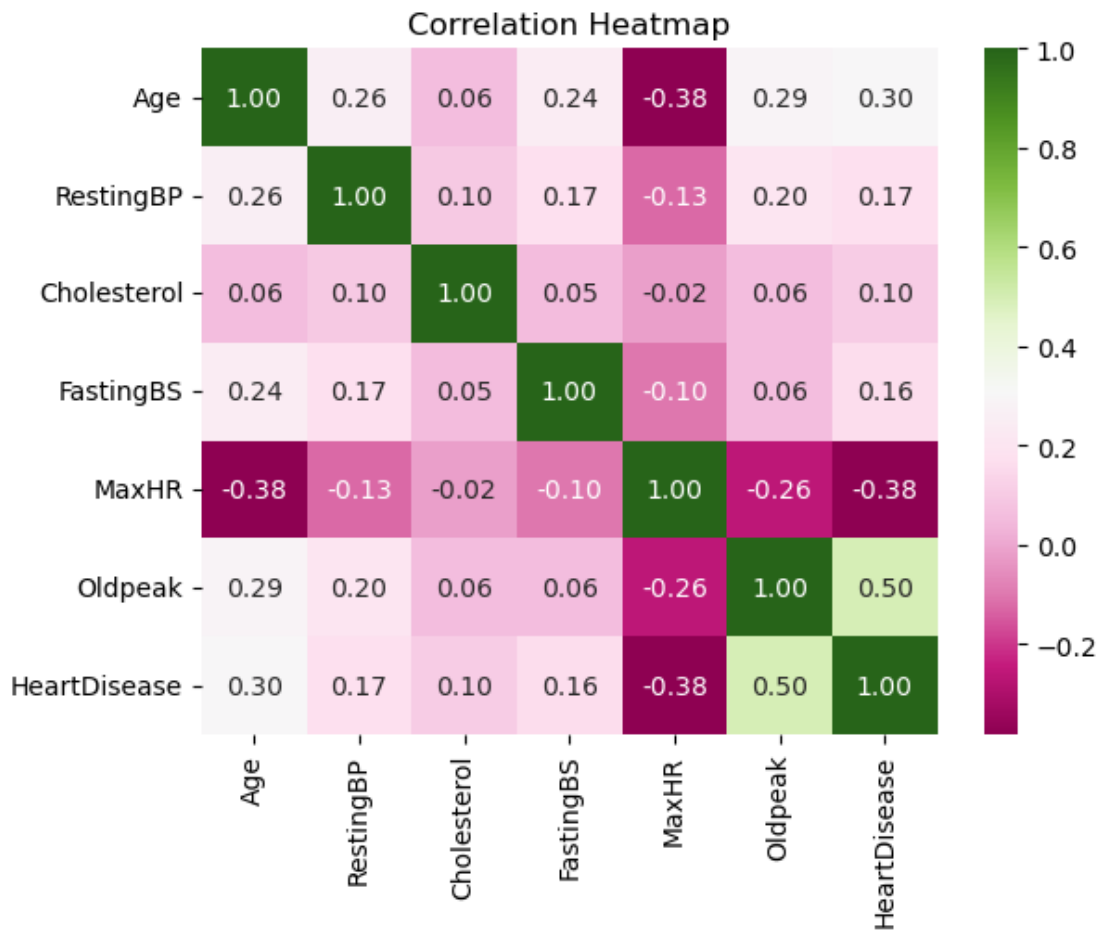
```
[398]:
```

	Age	RestingBP	Cholesterol	FastingBS	MaxHR	Oldpeak	\
Age	1.000000	0.259865	0.058758	0.241338	-0.382112	0.286006	
RestingBP	0.259865	1.000000	0.095939	0.173765	-0.125774	0.198575	
Cholesterol	0.058758	0.095939	1.000000	0.054012	-0.019856	0.058488	
FastingBS	0.241338	0.173765	0.054012	1.000000	-0.102710	0.055568	
MaxHR	-0.382112	-0.125774	-0.019856	-0.102710	1.000000	-0.259533	
Oldpeak	0.286006	0.198575	0.058488	0.055568	-0.259533	1.000000	

```
HeartDisease  0.298617  0.173242  0.103866  0.160594 -0.377212  0.495696
```

```
HeartDisease
Age          0.298617
RestingBP    0.173242
Cholesterol  0.103866
FastingBS    0.160594
MaxHR        -0.377212
Oldpeak      0.495696
HeartDisease 1.000000
```

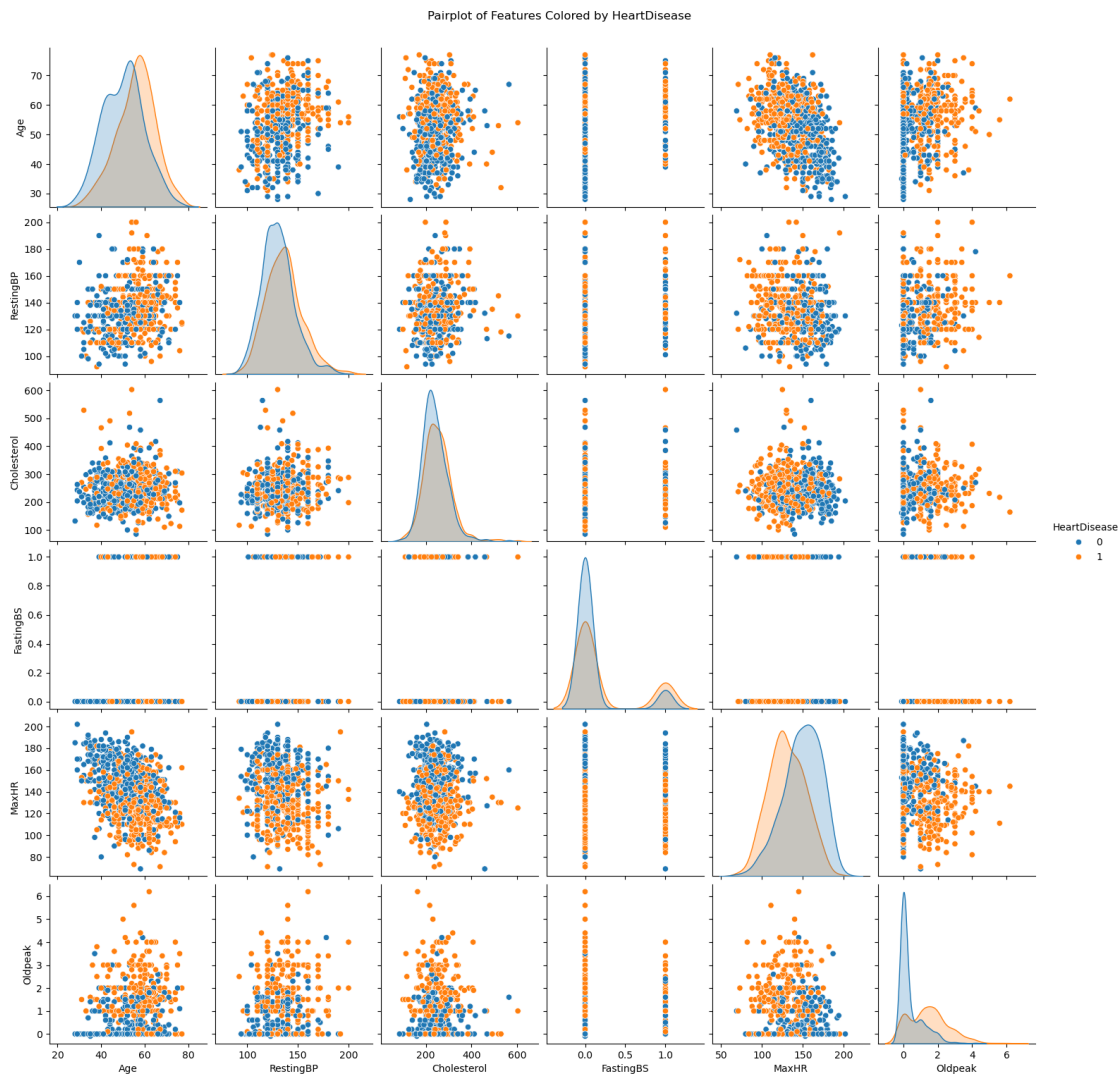
```
[399]: sns.heatmap(df.corr(numeric_only=True), annot=True, cmap='PiYG', fmt=".2f")
plt.title('Correlation Heatmap')
plt.show()
```



- most of the features are not having linear correlation
- (OldPeak vs Heart Diseases) showing a correlation value 0.5, which indicates that OldPeak might be significant in contributing the response variable Y

- The analysis of age versus maximum heart rate (MaxHR) and MaxHR versus heart diseases reveals a negative correlation of -0.38. This indicates that older individuals tend to have a lower MaxHR, while those with a lower MaxHR are more likely to have heart diseases.
- The analysis of age versus heart diseases shows a weak positive linear correlation of 0.3. This indicates that older individuals are more likely to have heart disease. Age could be another significant factor contributing to the occurrence of heart disease.

```
[400]: sns.pairplot(df, hue='HeartDisease')
plt.suptitle('Pairplot of Features Colored by HeartDisease', y=1.02)
plt.show()
```



5 Logistic Regression

```
[401]: from ISLP.models import (ModelSpec as MS,
                                summarize)

import statsmodels.api as sm
from sklearn.model_selection import StratifiedKFold
from sklearn.metrics import confusion_matrix, accuracy_score, precision_score, \
    recall_score, f1_score, roc_auc_score
from sklearn.model_selection import train_test_split
y = df['HeartDisease']
X = df.drop(columns=['HeartDisease'])

# One-hot encode categorical features
X_encoded = pd.get_dummies(X, drop_first=True).astype(int)
# Split the data
X_train, X_test, y_train, y_test = train_test_split(X_encoded, y, test_size=0.
    2, random_state=42)
```

```
[402]: X_train
```

```
[402]:
```

	Age	RestingBP	Cholesterol	FastingBS	MaxHR	Oldpeak	Sex_M	\
70	57	140	265	0	145	1	1	
164	52	140	225	0	140	0	0	
882	56	140	294	0	153	1	0	
265	54	160	305	0	175	0	1	
250	44	135	491	0	135	0	1	
..	...	...	...	...	...	...	...	
71	44	130	215	0	135	0	1	
106	48	120	254	0	110	0	0	
270	45	120	225	0	140	0	1	
607	53	144	300	1	128	1	1	
102	40	150	392	0	130	2	0	

	ChestPainType_ATA	ChestPainType_NAP	ChestPainType_TA	\
70	1	0	0	
164	1	0	0	
882	1	0	0	
265	1	0	0	
250	0	0	0	
..	...	...	...	
71	1	0	0	
106	0	0	0	
270	0	0	0	
607	0	0	0	
102	0	0	0	

	RestingECG_Normal	RestingECG_ST	ExerciseAngina_Y	ST_Slope_Flat	\
--	-------------------	---------------	------------------	---------------	---

70	0	1	1	1
164	1	0	0	0
882	0	0	0	1
265	1	0	0	0
250	1	0	0	1
..	...	...	...	...
71	1	0	0	0
106	0	1	0	0
270	1	0	0	0
607	0	1	1	1
102	1	0	0	1

	ST_Slope_Up
70	0
164	1
882	0
265	1
250	0
..	...
71	1
106	1
270	1
607	0
102	0

[596 rows x 15 columns]

```
[403]: y_train
```

```
[403]: 70      1
      164    0
      882    0
      265    0
      250    1
      ..
      71     0
      106    0
      270    0
      607    1
      102    1
      Name: HeartDisease, Length: 596, dtype: int64
```

```
[404]: design = MS(X_train.columns)
      X_train_transformed = design.fit_transform(X_train)
      y_train_transformed = y_train==1
      X_test_transformed = design.transform(X_test)
      y_test_transformed = y_test==1
```

```
[405]: X_train_transformed
```

```
[405]:      intercept  Age  RestingBP  Cholesterol  FastingBS  MaxHR  Oldpeak  Sex_M  \
70          1.0   57         140          265          0    145         1        1
164         1.0   52         140          225          0    140         0        0
882         1.0   56         140          294          0    153         1        0
265         1.0   54         160          305          0    175         0        1
250         1.0   44         135          491          0    135         0        1
..         ...   ...         ...          ...          ...   ...         ...
71          1.0   44         130          215          0    135         0        1
106         1.0   48         120          254          0    110         0        0
270         1.0   45         120          225          0    140         0        1
607         1.0   53         144          300          1    128         1        1
102         1.0   40         150          392          0    130         2        0
```

```
      ChestPainType_ATA  ChestPainType_NAP  ChestPainType_TA  \
70                   1                   0                   0
164                  1                   0                   0
882                  1                   0                   0
265                  1                   0                   0
250                  0                   0                   0
..                  ...                   ...                   ...
71                   1                   0                   0
106                  0                   0                   0
270                  0                   0                   0
607                  0                   0                   0
102                  0                   0                   0
```

```
      RestingECG_Normal  RestingECG_ST  ExerciseAngina_Y  ST_Slope_Flat  \
70                   0                   1                   1                   1
164                  1                   0                   0                   0
882                  0                   0                   0                   1
265                  1                   0                   0                   0
250                  1                   0                   0                   1
..                  ...                   ...                   ...                   ...
71                   1                   0                   0                   0
106                  0                   1                   0                   0
270                  1                   0                   0                   0
607                  0                   1                   1                   1
102                  1                   0                   0                   1
```

```
      ST_Slope_Up
70              0
164             1
882             0
265             1
250             0
```

```
..      ...
71      1
106     1
270     1
607     0
102     0
```

[596 rows x 16 columns]

```
[406]: y_train_transformed
```

```
[406]: 70      True
164     False
882     False
265     False
250      True
```

```
..
71     False
106     False
270     False
607      True
102      True
```

Name: HeartDisease, Length: 596, dtype: bool

```
[407]: glm = sm.GLM(y_train_transformed, X_train_transformed, family=sm.families.
      ↪Binomial())
model = glm.fit()
summarize(model)
```

```
[407]:
```

	coef	std err	z	P> z
intercept	-5.3510	2.008	-2.665	0.008
Age	0.0332	0.016	2.070	0.038
RestingBP	0.0158	0.008	1.930	0.054
Cholesterol	0.0008	0.002	0.363	0.717
FastingBS	0.0335	0.367	0.091	0.927
MaxHR	0.0005	0.006	0.080	0.936
Oldpeak	0.3926	0.169	2.329	0.020
Sex_M	1.5993	0.344	4.654	0.000
ChestPainType_ATA	-1.5292	0.384	-3.981	0.000
ChestPainType_NAP	-1.4902	0.331	-4.507	0.000
ChestPainType_TA	-1.9778	0.567	-3.490	0.000
RestingECG_Normal	-0.3201	0.315	-1.015	0.310
RestingECG_ST	-0.5188	0.439	-1.181	0.238
ExerciseAngina_Y	1.0388	0.294	3.539	0.000
ST_Slope_Flat	1.2281	0.608	2.019	0.044
ST_Slope_Up	-1.1282	0.668	-1.690	0.091

```
[408]: predictions_prob = model.predict(X_test_transformed)
       predictions = (predictions_prob >= 0.5).astype(int)
       predictions
```

```
[408]: 208    0
       259    0
       97    0
       148    0
       567    1
       ..
       726    0
       69    0
       694    0
       131    1
       44    1
       Length: 150, dtype: int64
```

```
[409]: y_test
```

```
[409]: 208    0
       259    0
       97    0
       148    0
       567    1
       ..
       726    0
       69    0
       694    0
       131    1
       44    1
       Name: HeartDisease, Length: 150, dtype: int64
```

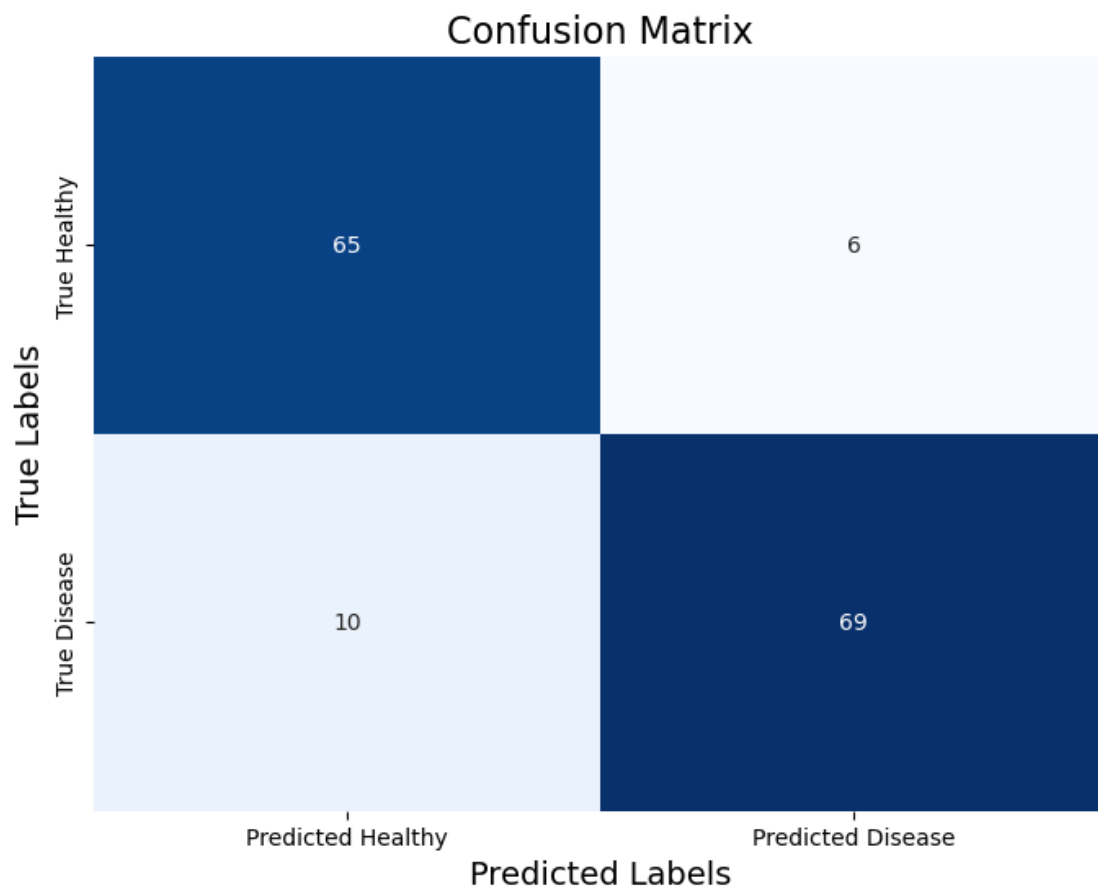
```
[410]: accuracy = accuracy_score(y_test, predictions)
       precision = precision_score(y_test, predictions)
       recall = recall_score(y_test, predictions)
       f1 = f1_score(y_test, predictions)
       roc_auc = roc_auc_score(y_test, predictions_prob) # Use
                 ↪ predicted_probabilities for AUC

       print(confusion_matrix(y_test, predictions))
       print(f"Accuracy: {accuracy:.2f}")
       print(f"Precision: {precision:.2f}")
       print(f"Recall: {recall:.2f}")
       print(f"F1 Score: {f1:.2f}")
       print(f"ROC AUC: {roc_auc:.2f}")
```

```
[[65  6]
```

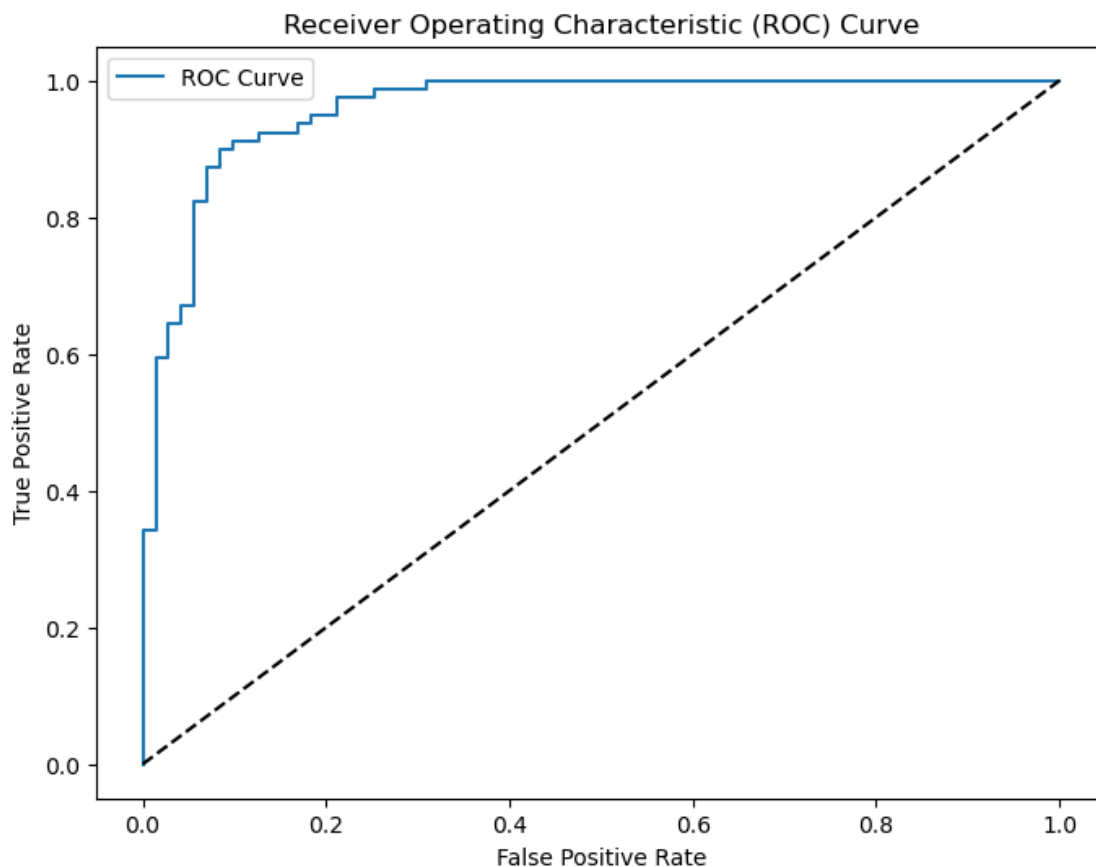
```
[10 69]]  
Accuracy: 0.89  
Precision: 0.92  
Recall: 0.87  
F1 Score: 0.90  
ROC AUC: 0.96
```

```
[411]: cm = confusion_matrix(y_test, predictions)  
  
# Plotting  
plt.figure(figsize=(8, 6))  
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', cbar=False,  
            xticklabels=['Predicted Healthy', 'Predicted Disease'],  
            yticklabels=['True Healthy', 'True Disease'])  
  
plt.title('Confusion Matrix', fontsize=16)  
plt.xlabel('Predicted Labels', fontsize=14)  
plt.ylabel('True Labels', fontsize=14)  
plt.show()
```



```
[412]: from sklearn.metrics import roc_curve
fpr, tpr, thresholds = roc_curve(y_test, predictions_prob)

plt.figure(figsize=(8, 6))
plt.plot(fpr, tpr, label='ROC Curve')
plt.plot([0, 1], [0, 1], 'k--')
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('Receiver Operating Characteristic (ROC) Curve')
plt.legend()
plt.show()
```

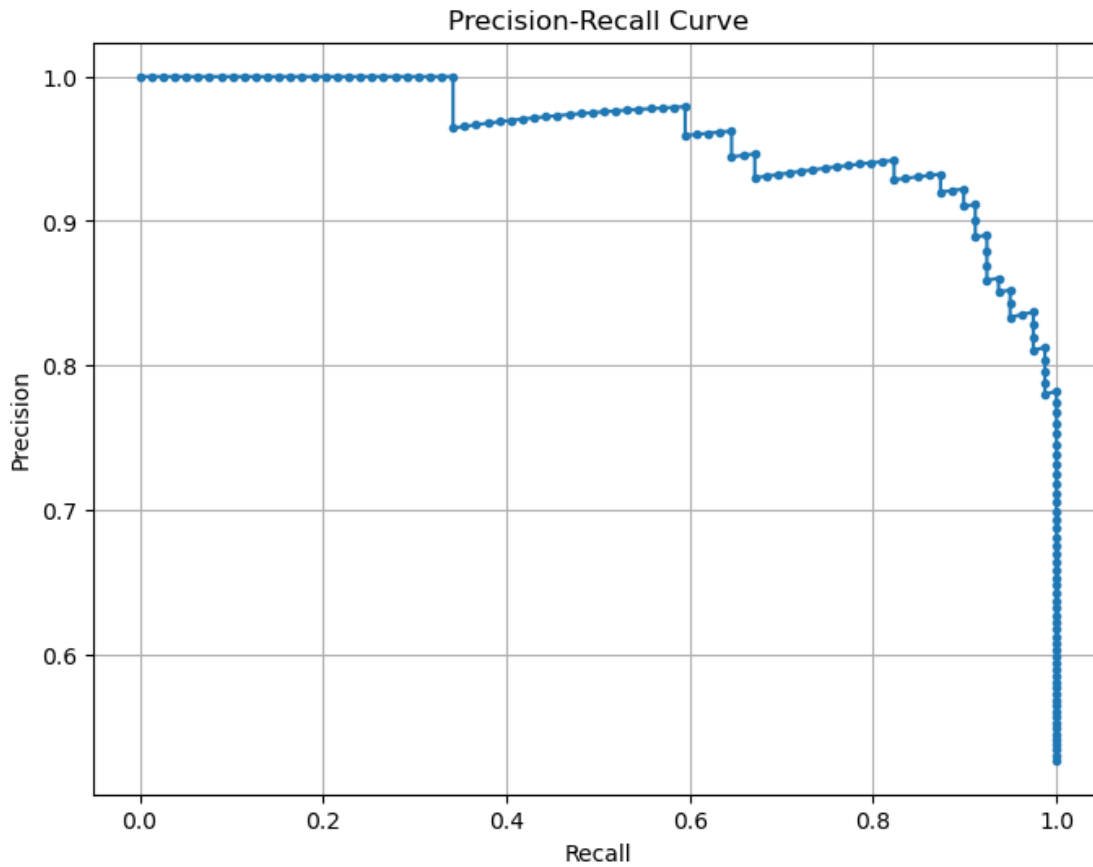


```
[413]: import matplotlib.pyplot as plt
from sklearn.metrics import precision_recall_curve

precision, recall, _ = precision_recall_curve(y_test, predictions_prob)

# Plotting the Precision-Recall curve
plt.figure(figsize=(8, 6))
```

```
plt.plot(recall, precision, marker='.')
plt.xlabel('Recall')
plt.ylabel('Precision')
plt.title('Precision-Recall Curve')
plt.grid()
plt.show()
```



```
[414]: # Assess the performance with Cross-validation
kf = StratifiedKFold(n_splits=10)

accuracy_scores = []
roc_auc_scores = []
f1_scores = []
precision_scores = []
recall_scores = []

for train_index, val_index in kf.split(X_train_transformed, y_train_transformed):
```



```

X_train, X_val = X_train_transformed.iloc[train_index], X_train_transformed.
↪iloc[val_index]
y_train, y_val = y_train_transformed.iloc[train_index], y_train_transformed.
↪iloc[val_index]

# Fit the model
glm = sm.GLM(y_train, X_train, family=sm.families.Binomial())
results = glm.fit()

# Predict
y_pred_proba = results.predict(X_val)
y_pred = (y_pred_proba > 0.5).astype(int)

# Calculate metrics
accuracy = accuracy_score(y_val, y_pred)
roc_auc = roc_auc_score(y_val, y_pred_proba)
f1 = f1_score(y_val, y_pred)
precision = precision_score(y_val, y_pred)
recall = recall_score(y_val, y_pred)

accuracy_scores.append(accuracy)
roc_auc_scores.append(roc_auc)
f1_scores.append(f1)
precision_scores.append(precision)
recall_scores.append(recall)

print(f'Average Accuracy: {np.mean(accuracy_scores):.4f}')
print(f'Average ROC AUC: {np.mean(roc_auc_scores):.4f}')
print(f'Average F1 Score: {np.mean(f1_scores):.4f}')
print(f'Average Precision: {np.mean(precision_scores):.4f}')
print(f'Average Recall: {np.mean(recall_scores):.4f}')

```

```

Average Accuracy: 0.8371
Average ROC AUC: 0.9159
Average F1 Score: 0.8272
Average Precision: 0.8220
Average Recall: 0.8378

```

main_rf

November 29, 2025

1 Random Forest

```
[ ]: from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, precision_score, recall_score,
    f1_score, roc_auc_score, roc_curve, auc, precision_recall_curve,
    average_precision_score

X_train_rf, X_test_rf, y_train_rf, y_test_rf = train_test_split(X_encoded, y,
    test_size=0.2, random_state=42)
```

```
[ ]: from sklearn.model_selection import GridSearchCV

param_list = list(np.arange(1, 20))
param_list.append(None)

param_grid = {
    'max_depth': param_list,
    'n_estimators': [50, 100, 150, 200, 250, 300, 350, 400, 450, 500]
}

rf = RandomForestClassifier(random_state=42)
grid_rf = GridSearchCV(estimator=rf, param_grid=param_grid, cv=5,
    scoring='accuracy', n_jobs=-1)
grid_rf.fit(X_train_rf, y_train_rf)

print(f"Best Max Depth: {grid_rf.best_params_['max_depth']}")
print(f"Best N Estimators: {grid_rf.best_params_['n_estimators']}")
print(f"Best Cross-Validation Accuracy: {grid_rf.best_score_:.4f}")
```

Best Max Depth: 11

Best N Estimators: 200

Best Cross-Validation Accuracy: 0.8624

```
[ ]: y_pred_rf = grid_rf.predict(X_test_rf)
y_pred_rf_prob = grid_rf.predict_proba(X_test_rf)[: , 1]

# Evaluate the model
accuracy_rf = accuracy_score(y_test_rf, y_pred_rf)
```

```

print(f"Accuracy: {accuracy_rf:.4f}")

precision_rf = precision_score(y_test_rf, y_pred_rf)
print(f"Precision: {precision_rf:.4f}")

recall_rf = recall_score(y_test_rf, y_pred_rf)
print(f"Recall: {recall_rf:.4f}")

f1_rf = f1_score(y_test_rf, y_pred_rf)
print(f"F1 Score: {f1_rf:.4f}")

roc_auc = roc_auc_score(y_test_rf, y_pred_rf_prob)
print(f"ROC AUC: {roc_auc:.4f}")

```

Accuracy: 0.9067
 Precision: 0.9333
 Recall: 0.8861
 F1 Score: 0.9091
 ROC AUC: 0.9553

```

[ ]: cm = confusion_matrix(y_test_rf, y_pred_rf)
    tn, fp, fn, tp = cm.ravel()

print(f"True Positive (TP): {tp}")
print(f"False Positive (FP): {fp}")
print(f"True Negative (TN): {tn}")
print(f"False Negative (FN): {fn}")

print(f"Total Test Samples: {len(y_test)}")

```

True Positive (TP): 70
 False Positive (FP): 5
 True Negative (TN): 66
 False Negative (FN): 9
 Total Test Samples: 150

```

[ ]: fpr, tpr, thresholds = roc_curve(y_test_rf, y_pred_rf_prob)

precision, recall, thresholds = precision_recall_curve(y_test_rf,
    ↪ y_pred_rf_prob)
average_precision = average_precision_score(y_test_rf, y_pred_rf_prob)

fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12, 6))

ax1.plot(fpr, tpr, label=f'ROC Curve (area = {roc_auc:.4f})')
ax1.plot([0, 1], [0, 1], 'k--')
ax1.set_xlabel('False Positive Rate')

```

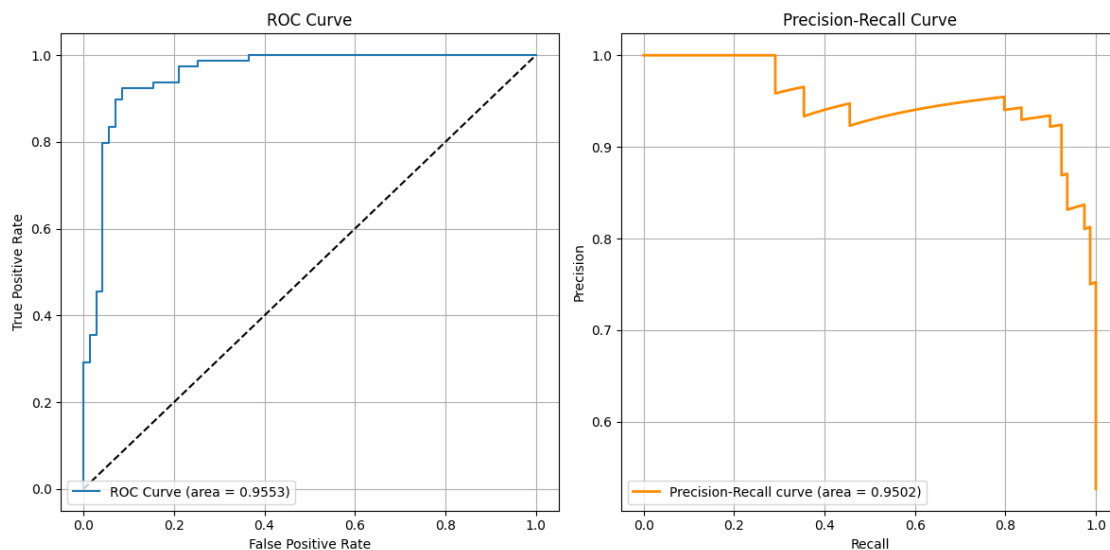
```

ax1.set_ylabel('True Positive Rate')
ax1.set_title('ROC Curve')
ax1.legend(loc="lower left")
ax1.grid(True)

ax2.plot(recall, precision, lw=2, color='darkorange', label=f'Precision-Recall_
↳curve (area = {average_precision:.4f})')
ax2.set_xlabel('Recall')
ax2.set_ylabel('Precision')
ax2.set_title('Precision-Recall Curve')
ax2.legend(loc="lower left")
ax2.grid(True)

plt.tight_layout()
plt.show()

```



```

[ ]: best_rf = grid_rf.best_estimator_

important_features = pd.DataFrame({
    'Feature': X_train_rf.columns,
    'Importance': best_rf.feature_importances_
}).sort_values(by='Importance', ascending=False)

important_features

```

```

[ ]:
      Feature  Importance
14  ST_Slope_Up    0.157258
4    MaxHR        0.125839

```

12	ExerciseAngina_Y	0.113164
13	ST_Slope_Flat	0.109014
0	Age	0.096027
1	RestingBP	0.082422
2	Cholesterol	0.080508
5	Oldpeak	0.078007
6	Sex_M	0.036866
7	ChestPainType_ATA	0.033517
8	ChestPainType_NAP	0.025571
10	RestingECG_Normal	0.020070
9	ChestPainType_TA	0.016788
3	FastingBS	0.012562
11	RestingECG_ST	0.012388

main_gbm

November 29, 2025

1 Gradient Boosting Machine

```
[1]: import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.ensemble import GradientBoostingClassifier
from sklearn.metrics import roc_curve, roc_auc_score, accuracy_score, \
    classification_report
import matplotlib.pyplot as plt

# --- 1. Load Data ---
# Load the dataset
df = pd.read_csv('heart-cleaned.csv')

# --- 2. Data Preprocessing (One-Hot Encoding) ---
# Identify categorical columns for encoding
categorical_cols = ['Sex', 'ChestPainType', 'RestingECG', 'ExerciseAngina', \
    'ST_Slope']
df_encoded = pd.get_dummies(df, columns=categorical_cols, drop_first=True)

# Define features (X) and target (y)
X = df_encoded.drop('HeartDisease', axis=1)
y = df_encoded['HeartDisease']

# --- 3. Train-Test Split ---
# Split the data into 70% training and 30% testing, using stratify to maintain \
    class balance
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, \
    random_state=42, stratify=y)

# --- 4. Gradient Boosting Model (GBM) Training ---
# Initialize and train the classifier
gbm_model = GradientBoostingClassifier(random_state=42)
gbm_model.fit(X_train, y_train)

# --- 5. Model Evaluation and ROC/AUC Calculation ---

# Get predicted probabilities for the positive class (HeartDisease = 1)
```

```

y_prob = gbm_model.predict_proba(X_test)[: , 1]

# Calculate ROC curve metrics (False Positive Rate, True Positive Rate)
fpr, tpr, thresholds = roc_curve(y_test, y_prob)

# Calculate Area Under the Curve (AUC)
roc_auc = roc_auc_score(y_test, y_prob)

# Get hard predictions for standard metrics
y_pred = gbm_model.predict(X_test)

# --- 6. Print Results ---
print("--- Gradient Boosting Model Results ---")
print(f"Area Under the Curve (AUC): {roc_auc:.4f}")
print(f"Accuracy: {accuracy_score(y_test, y_pred):.4f}")
print("\nClassification Report:\n", classification_report(y_test, y_pred))

# --- 7. Print top feature importances ---
feature_importance = pd.Series(gbm_model.feature_importances_, index=X_train.
    ↪columns).sort_values(ascending=False).head(10)
print("\nTop 10 Feature Importances:\n", feature_importance)

# --- 8. Plot ROC Curve ---
plt.figure(figsize=(8, 6))
plt.plot(fpr, tpr, color='darkorange', lw=2, label='ROC curve (area = %0.4f)' %
    ↪roc_auc)
plt.plot([0, 1], [0, 1], color='navy', lw=2, linestyle='--', label='Random
    ↪Classifier')
plt.xlim([0.0, 1.0])
plt.ylim([0.0, 1.05])
plt.xlabel('False Positive Rate (1 - Specificity)')
plt.ylabel('True Positive Rate (Sensitivity/Recall)')
plt.title('Receiver Operating Characteristic (ROC) Curve for GBM')
plt.legend(loc="lower right")
plt.grid(True)
plt.show()

```

```

--- Gradient Boosting Model Results ---
Area Under the Curve (AUC): 0.9292
Accuracy: 0.8571

```

Classification Report:

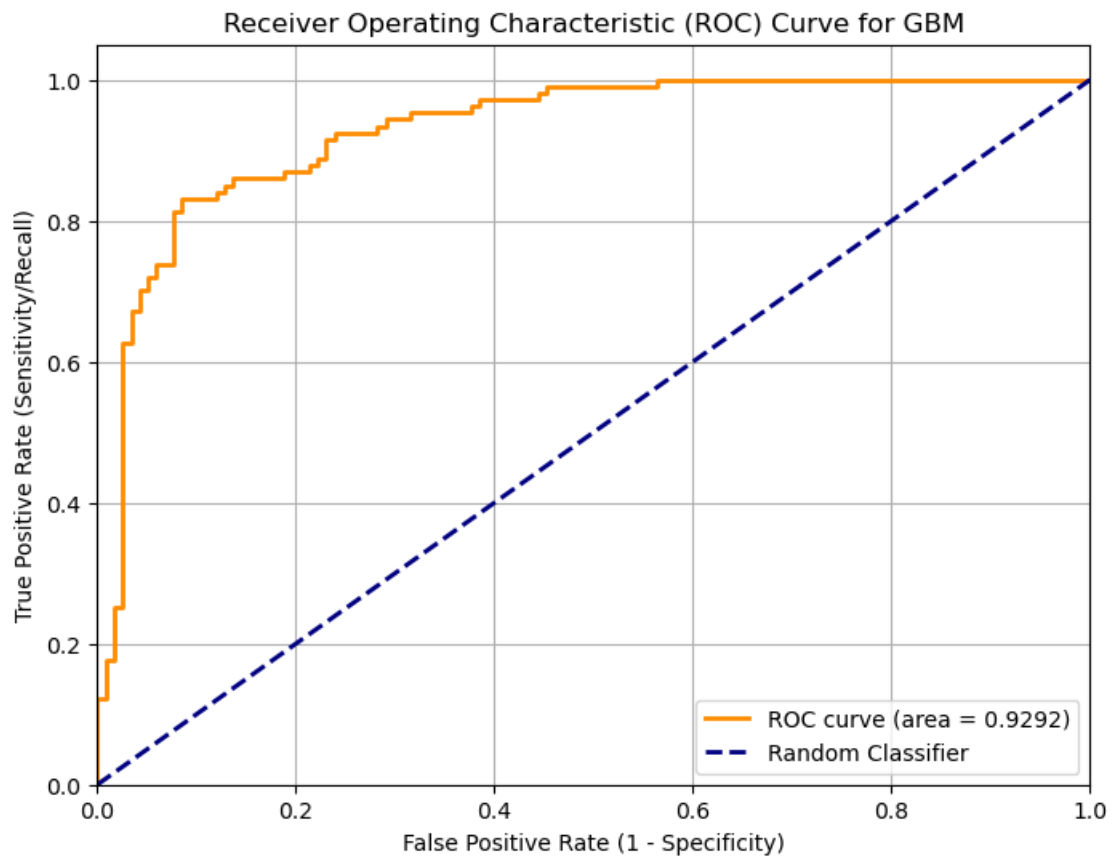
	precision	recall	f1-score	support
0	0.86	0.86	0.86	117
1	0.85	0.85	0.85	107

accuracy			0.86	224
macro avg	0.86	0.86	0.86	224
weighted avg	0.86	0.86	0.86	224

Top 10 Feature Importances:

ST_Slope_Up	0.484202
MaxHR	0.101784
Oldpeak	0.097017
ExerciseAngina_Y	0.074035
Sex_M	0.068790
Age	0.051784
RestingBP	0.029303
Cholesterol	0.022523
ST_Slope_Flat	0.021423
ChestPainType_ATA	0.015787

dtype: float64



main-svm

November 29, 2025

1 SVM

```
[27]: # Import necessary libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline
from sklearn.svm import SVC
from sklearn.metrics import (
    accuracy_score,
    confusion_matrix,
    classification_report,
    roc_curve,
    roc_auc_score,
)
```

```
[28]: # Load the cleaned dataset
df = pd.read_csv("heart-cleaned.csv")
print(f"Dataset shape: {df.shape}")
print(f"\nFirst few rows:")
df.head()
```

Dataset shape: (746, 12)

First few rows:

```
[28]:
```

	Age	Sex	ChestPainType	RestingBP	Cholesterol	FastingBS	RestingECG	MaxHR	\
0	40	M	ATA	140	289	0	Normal	172	
1	49	F	NAP	160	180	0	Normal	156	
2	37	M	ATA	130	283	0	ST	98	
3	48	F	ASY	138	214	0	Normal	108	
4	54	M	NAP	150	195	0	Normal	122	

	ExerciseAngina	Oldpeak	ST_Slope	HeartDisease
0	N	0.0	Up	0

1	N	1.0	Flat	1
2	N	0.0	Up	0
3	Y	1.5	Flat	1
4	N	0.0	Up	0

```
[29]: # Separate features and target
X = df.drop(columns=["HeartDisease"])
y = df["HeartDisease"]

# Identify numerical and categorical columns
numerical_cols = ["Age", "RestingBP", "Cholesterol", "FastingBS", "MaxHR", "Oldpeak"]
categorical_cols = ["Sex", "ChestPainType", "RestingECG", "ExerciseAngina", "ST_Slope"]

print(f"Numerical columns: {numerical_cols}")
print(f"Categorical columns: {categorical_cols}")
print(f"\nTarget distribution:")
print(y.value_counts())
```

Numerical columns: ['Age', 'RestingBP', 'Cholesterol', 'FastingBS', 'MaxHR', 'Oldpeak']

Categorical columns: ['Sex', 'ChestPainType', 'RestingECG', 'ExerciseAngina', 'ST_Slope']

Target distribution:

HeartDisease

0 390

1 356

Name: count, dtype: int64

```
[30]: # One-hot encode categorical variables
X_encoded = pd.get_dummies(X, columns=categorical_cols, drop_first=True)
print(f"Shape after encoding: {X_encoded.shape}")
print(f"\nFeature names:")
print(X_encoded.columns.tolist())
```

Shape after encoding: (746, 15)

Feature names:

['Age', 'RestingBP', 'Cholesterol', 'FastingBS', 'MaxHR', 'Oldpeak', 'Sex_M', 'ChestPainType_ATA', 'ChestPainType_NAP', 'ChestPainType_TA', 'RestingECG_Normal', 'RestingECG_ST', 'ExerciseAngina_Y', 'ST_Slope_Flat', 'ST_Slope_Up']

```
[31]: # Split the data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(
    X_encoded, y, test_size=0.2, random_state=42, stratify=y)
```

```
)

print(f"Training set size: {X_train.shape[0]}")
print(f"Test set size: {X_test.shape[0]}")
print(f"\nTraining target distribution:")
print(y_train.value_counts())
print(f"\nTest target distribution:")
print(y_test.value_counts())
```

Training set size: 596

Test set size: 150

Training target distribution:

HeartDisease

0 312

1 284

Name: count, dtype: int64

Test target distribution:

HeartDisease

0 78

1 72

Name: count, dtype: int64

```
[32]: # Define function for hyperparameter tuning
def tune_clf_hyperparameters(
    pipeline, param_grid, X_train, y_train, cv=5, scoring="accuracy"
):
    grid_search = GridSearchCV(
        pipeline, param_grid, cv=cv, scoring=scoring, n_jobs=-1, verbose=1
    )
    grid_search.fit(X_train, y_train)

    print(f"\nBest cross-validation score: {grid_search.best_score_:.4f}")
    print(f"Best parameters: {grid_search.best_params_}\n")

    return grid_search.best_estimator_, grid_search.best_params_
```

```
[33]: # Create SVM pipeline with standardization
svm_pipeline = Pipeline([("scaler", StandardScaler()), ("svm",
    ↪SVC(random_state=42))])

# Configure hyperparameter grid
param_grid_svm = {
    "svm__C": [0.001, 0.005, 0.01, 0.05, 0.1, 1, 10, 20],
    "svm__kernel": ["linear", "rbf", "poly"],
    "svm__gamma": ["scale", "auto", 0.1, 0.5, 1, 5],
```

```

        "svm__degree": [2, 3, 4],
    }

    # Perform GridSearchCV to get all results
    grid_search = GridSearchCV(
        svm_pipeline, param_grid_svm, cv=5, scoring="accuracy", n_jobs=-1, verbose=1
    )
    grid_search.fit(X_train, y_train)

    # Get best model and parameters
    best_svm = grid_search.best_estimator_
    best_svm_hyperparams = grid_search.best_params_

    print("\n" + "=" * 70)
    print("SVM Optimal Hyperparameters:")
    print("=" * 70)
    for param, value in best_svm_hyperparams.items():
        print(f"    {param}: {value}")
    print(f"\nBest cross-validation score: {grid_search.best_score_:.4f}")
    print("=" * 70)

    # Create DataFrame with all results
    results_df = pd.DataFrame(grid_search.cv_results_)

    # Select relevant columns and sort by mean test score
    results_display = results_df[
        [
            "param_svm__C",
            "param_svm__kernel",
            "param_svm__gamma",
            "param_svm__degree",
            "mean_test_score",
            "std_test_score",
        ]
    ].copy()

    # Sort by mean test score in descending order
    results_display = results_display.sort_values("mean_test_score",
        ↪ascending=False)

    # Rename columns for better readability
    results_display.columns = ["C", "Kernel", "Gamma", "Degree", "Mean CV Score",
        ↪"Std CV Score"]

    print("\n" + "=" * 100)
    print("ALL HYPERPARAMETER COMBINATIONS RESULTS (Sorted by Mean CV Score)")
    print("=" * 100)

```

```

print(f"\nTotal combinations tested: {len(results_display)}")
print(f"\nTop 20 combinations:")
print("-" * 100)
print(results_display.head(20).to_string(index=False))
print("\n" + "-" * 100)

# Display all results (can be scrolled in notebook)
print(f"\n\nALL {len(results_display)} COMBINATIONS (Complete Results):")
print("=" * 100)
pd.set_option('display.max_rows', None)
pd.set_option('display.max_columns', None)
pd.set_option('display.width', None)
pd.set_option('display.max_colwidth', None)
print(results_display.to_string(index=False))
pd.reset_option('display.max_rows')
pd.reset_option('display.max_columns')
pd.reset_option('display.width')
pd.reset_option('display.max_colwidth')
print("\n" + "=" * 100)

# Group results by kernel to show summary
print("\n" + "=" * 100)
print("SUMMARY BY KERNEL TYPE")
print("=" * 100)
for kernel in ["linear", "rbf", "poly"]:
    kernel_results = results_display[results_display["Kernel"] == kernel]
    if len(kernel_results) > 0:
        print(f"\n{kernel.upper()} Kernel:")
        print(f"  Best score: {kernel_results['Mean CV Score'].max():.4f}")
        print(f"  Worst score: {kernel_results['Mean CV Score'].min():.4f}")
        print(f"  Average score: {kernel_results['Mean CV Score'].mean():.4f}")
        print(f"  Number of combinations: {len(kernel_results)}")
        best_kernel_combo = kernel_results.iloc[0]
        print(f"  Best combination: C={best_kernel_combo['C']},  

↳ Gamma={best_kernel_combo['Gamma']}, Degree={best_kernel_combo['Degree']}")

print("\n" + "=" * 100)

```

Fitting 5 folds for each of 432 candidates, totalling 2160 fits

=====

SVM Optimal Hyperparameters:

=====

```

svm__C: 0.05
svm__degree: 2
svm__gamma: scale
svm__kernel: rbf

```

Best cross-validation score: 0.8674

=====

=====

ALL HYPERPARAMETER COMBINATIONS RESULTS (Sorted by Mean CV Score)

=====

Total combinations tested: 432

Top 20 combinations:

C	Kernel	Gamma	Degree	Mean CV Score	Std CV Score
0.050	rbf	auto	2	0.867409	0.020383
0.050	rbf	scale	3	0.867409	0.020383
0.050	rbf	auto	3	0.867409	0.020383
0.050	rbf	scale	4	0.867409	0.020383
0.050	rbf	auto	4	0.867409	0.020383
0.050	rbf	scale	2	0.867409	0.020383
0.050	rbf	0.1	2	0.862381	0.024289
0.050	rbf	0.1	3	0.862381	0.024289
0.050	rbf	0.1	4	0.862381	0.024289
0.100	rbf	scale	2	0.862381	0.013946
0.100	rbf	auto	4	0.862381	0.013946
0.100	rbf	auto	2	0.862381	0.013946
0.100	rbf	scale	3	0.862381	0.013946
0.100	rbf	scale	4	0.862381	0.013946
0.100	rbf	auto	3	0.862381	0.013946
0.100	poly	auto	3	0.859048	0.014568
0.100	poly	scale	3	0.859048	0.014568
0.001	poly	0.5	3	0.859034	0.021013
0.001	linear	scale	4	0.857367	0.019985
0.001	linear	5	3	0.857367	0.019985

ALL 432 COMBINATIONS (Complete Results):

=====

C	Kernel	Gamma	Degree	Mean CV Score	Std CV Score
0.050	rbf	auto	2	0.867409	0.020383
0.050	rbf	scale	3	0.867409	0.020383
0.050	rbf	auto	3	0.867409	0.020383
0.050	rbf	scale	4	0.867409	0.020383

0.050	rbf	auto	4	0.867409	0.020383
0.050	rbf	scale	2	0.867409	0.020383
0.050	rbf	0.1	2	0.862381	0.024289
0.050	rbf	0.1	3	0.862381	0.024289
0.050	rbf	0.1	4	0.862381	0.024289
0.100	rbf	scale	2	0.862381	0.013946
0.100	rbf	auto	4	0.862381	0.013946
0.100	rbf	auto	2	0.862381	0.013946
0.100	rbf	scale	3	0.862381	0.013946
0.100	rbf	scale	4	0.862381	0.013946
0.100	rbf	auto	3	0.862381	0.013946
0.100	poly	auto	3	0.859048	0.014568
0.100	poly	scale	3	0.859048	0.014568
0.001	poly	0.5	3	0.859034	0.021013
0.001	linear	scale	4	0.857367	0.019985
0.001	linear	5	3	0.857367	0.019985
0.001	linear	auto	2	0.857367	0.019985
0.001	linear	5	4	0.857367	0.019985
0.001	linear	0.1	2	0.857367	0.019985
0.001	linear	1	4	0.857367	0.019985
0.100	rbf	0.1	4	0.857367	0.015164
0.001	linear	0.5	4	0.857367	0.019985
0.001	linear	0.5	2	0.857367	0.019985
0.001	linear	0.1	4	0.857367	0.019985
0.001	linear	auto	4	0.857367	0.019985
0.100	rbf	0.1	2	0.857367	0.015164
0.001	linear	scale	3	0.857367	0.019985
0.001	linear	scale	2	0.857367	0.019985
0.001	linear	5	2	0.857367	0.019985
0.001	linear	1	3	0.857367	0.019985
0.100	rbf	0.1	3	0.857367	0.015164
0.001	linear	0.5	3	0.857367	0.019985
0.001	linear	auto	3	0.857367	0.019985
0.001	linear	1	2	0.857367	0.019985
0.001	linear	0.1	3	0.857367	0.019985
0.100	linear	0.1	4	0.857353	0.023920
0.100	linear	5	4	0.857353	0.023920
0.100	linear	0.5	3	0.857353	0.023920
0.100	linear	1	4	0.857353	0.023920
0.100	linear	0.5	4	0.857353	0.023920
0.100	linear	0.1	3	0.857353	0.023920
0.100	linear	5	3	0.857353	0.023920
0.100	linear	scale	3	0.857353	0.023920
0.100	linear	auto	2	0.857353	0.023920
0.100	linear	auto	3	0.857353	0.023920
0.100	linear	auto	4	0.857353	0.023920
0.100	linear	0.1	2	0.857353	0.023920
0.100	linear	scale	4	0.857353	0.023920

0.100	linear	0.5	2	0.857353	0.023920
0.100	linear	1	2	0.857353	0.023920
0.100	linear	5	2	0.857353	0.023920
0.100	linear	1	3	0.857353	0.023920
0.100	linear	scale	2	0.857353	0.023920
20.000	linear	0.5	3	0.857339	0.025129
1.000	linear	1	4	0.857339	0.025129
20.000	linear	5	3	0.857339	0.025129
10.000	linear	scale	2	0.857339	0.025129
10.000	linear	auto	2	0.857339	0.025129
20.000	linear	1	3	0.857339	0.025129
10.000	linear	0.1	2	0.857339	0.025129
10.000	linear	1	4	0.857339	0.025129
10.000	linear	0.5	2	0.857339	0.025129
10.000	linear	1	2	0.857339	0.025129
10.000	linear	5	2	0.857339	0.025129
10.000	linear	scale	3	0.857339	0.025129
10.000	linear	auto	3	0.857339	0.025129
10.000	linear	0.1	3	0.857339	0.025129
20.000	linear	0.1	3	0.857339	0.025129
10.000	linear	0.5	3	0.857339	0.025129
10.000	linear	0.5	4	0.857339	0.025129
20.000	linear	auto	3	0.857339	0.025129
20.000	linear	scale	3	0.857339	0.025129
10.000	linear	1	3	0.857339	0.025129
10.000	linear	5	4	0.857339	0.025129
20.000	linear	5	2	0.857339	0.025129
10.000	linear	5	3	0.857339	0.025129
10.000	linear	0.1	4	0.857339	0.025129
20.000	linear	1	2	0.857339	0.025129
20.000	linear	scale	2	0.857339	0.025129
20.000	linear	0.5	2	0.857339	0.025129
10.000	linear	scale	4	0.857339	0.025129
20.000	linear	0.1	2	0.857339	0.025129
20.000	linear	auto	2	0.857339	0.025129
1.000	linear	0.5	3	0.857339	0.025129
1.000	linear	5	4	0.857339	0.025129
10.000	linear	auto	4	0.857339	0.025129
20.000	linear	scale	4	0.857339	0.025129
1.000	linear	1	3	0.857339	0.025129
1.000	linear	1	2	0.857339	0.025129
20.000	linear	0.5	4	0.857339	0.025129
1.000	linear	0.5	2	0.857339	0.025129
1.000	linear	auto	3	0.857339	0.025129
1.000	linear	0.1	2	0.857339	0.025129
1.000	linear	auto	2	0.857339	0.025129
20.000	linear	1	4	0.857339	0.025129
1.000	linear	scale	3	0.857339	0.025129

20.000	linear	0.1	4	0.857339	0.025129
1.000	linear	scale	2	0.857339	0.025129
1.000	linear	5	2	0.857339	0.025129
1.000	linear	5	3	0.857339	0.025129
1.000	linear	auto	4	0.857339	0.025129
1.000	linear	0.5	4	0.857339	0.025129
1.000	linear	0.1	4	0.857339	0.025129
20.000	linear	5	4	0.857339	0.025129
1.000	linear	0.1	3	0.857339	0.025129
1.000	linear	scale	4	0.857339	0.025129
20.000	linear	auto	4	0.857339	0.025129
0.050	poly	0.1	3	0.855686	0.017277
1.000	poly	scale	3	0.855686	0.022901
1.000	poly	auto	3	0.855686	0.022901
0.100	poly	0.1	3	0.855672	0.018926
0.050	linear	0.1	3	0.852311	0.021213
0.050	linear	scale	3	0.852311	0.021213
0.050	linear	0.5	2	0.852311	0.021213
0.050	linear	0.5	3	0.852311	0.021213
0.050	linear	0.1	2	0.852311	0.021213
0.050	linear	1	3	0.852311	0.021213
0.050	linear	auto	3	0.852311	0.021213
0.050	linear	auto	2	0.852311	0.021213
0.050	linear	0.5	4	0.852311	0.021213
0.050	linear	5	3	0.852311	0.021213
0.050	linear	1	2	0.852311	0.021213
0.050	linear	5	4	0.852311	0.021213
0.050	linear	1	4	0.852311	0.021213
0.050	linear	scale	2	0.852311	0.021213
0.050	linear	5	2	0.852311	0.021213
0.050	linear	0.1	4	0.852311	0.021213
0.050	linear	auto	4	0.852311	0.021213
0.050	linear	scale	4	0.852311	0.021213
0.005	poly	0.5	3	0.852311	0.017572
1.000	rbf	auto	2	0.850658	0.009973
1.000	rbf	auto	3	0.850658	0.009973
1.000	rbf	scale	2	0.850658	0.009973
1.000	rbf	scale	3	0.850658	0.009973
1.000	rbf	scale	4	0.850658	0.009973
1.000	rbf	auto	4	0.850658	0.009973
0.010	linear	0.1	4	0.848992	0.016814
0.010	linear	1	4	0.848992	0.016814
0.010	linear	auto	4	0.848992	0.016814
0.010	linear	0.5	4	0.848992	0.016814
0.010	linear	scale	2	0.848992	0.016814
0.010	linear	scale	4	0.848992	0.016814
0.010	linear	5	3	0.848992	0.016814
0.010	linear	1	3	0.848992	0.016814

0.010	linear	0.5	3	0.848992	0.016814
0.010	linear	0.1	3	0.848992	0.016814
0.010	linear	auto	3	0.848992	0.016814
0.010	linear	scale	3	0.848992	0.016814
0.010	linear	5	2	0.848992	0.016814
0.010	linear	1	2	0.848992	0.016814
0.010	linear	0.5	2	0.848992	0.016814
0.010	linear	0.1	2	0.848992	0.016814
0.010	linear	auto	2	0.848992	0.016814
0.010	linear	5	4	0.848992	0.016814
0.010	poly	0.5	3	0.848964	0.017845
0.001	poly	1	3	0.847283	0.015677
1.000	poly	0.1	3	0.847283	0.015677
0.050	poly	auto	3	0.845630	0.019649
0.050	poly	scale	3	0.845630	0.019649
1.000	rbf	0.1	2	0.845588	0.026590
1.000	rbf	0.1	3	0.845588	0.026590
1.000	rbf	0.1	4	0.845588	0.026590
0.005	linear	auto	2	0.843964	0.019561
0.005	linear	0.1	4	0.843964	0.019561
0.005	linear	1	4	0.843964	0.019561
0.005	linear	5	4	0.843964	0.019561
0.005	linear	auto	4	0.843964	0.019561
0.005	linear	scale	4	0.843964	0.019561
0.005	linear	5	3	0.843964	0.019561
0.005	linear	0.5	3	0.843964	0.019561
0.005	linear	0.1	3	0.843964	0.019561
0.005	linear	auto	3	0.843964	0.019561
0.005	linear	scale	3	0.843964	0.019561
0.005	linear	1	3	0.843964	0.019561
0.005	linear	0.5	4	0.843964	0.019561
0.005	linear	scale	2	0.843964	0.019561
0.005	linear	1	2	0.843964	0.019561
0.005	linear	0.1	2	0.843964	0.019561
0.005	linear	0.5	2	0.843964	0.019561
0.005	linear	5	2	0.843964	0.019561
10.000	poly	auto	3	0.835532	0.011856
10.000	poly	scale	3	0.835532	0.011856
0.100	poly	0.1	4	0.825490	0.016419
10.000	rbf	auto	3	0.825462	0.022441
10.000	rbf	auto	2	0.825462	0.022441
10.000	rbf	auto	4	0.825462	0.022441
10.000	rbf	scale	2	0.825462	0.022441
10.000	rbf	scale	3	0.825462	0.022441
10.000	rbf	scale	4	0.825462	0.022441
10.000	rbf	0.1	4	0.818768	0.024284
10.000	rbf	0.1	2	0.818768	0.024284
10.000	rbf	0.1	3	0.818768	0.024284

1.000	poly	scale	4	0.815420	0.026673
1.000	poly	auto	4	0.815420	0.026673
1.000	rbf	0.5	4	0.812017	0.032878
1.000	rbf	0.5	3	0.812017	0.032878
1.000	rbf	0.5	2	0.812017	0.032878
20.000	rbf	scale	2	0.808697	0.027986
20.000	rbf	auto	3	0.808697	0.027986
20.000	rbf	scale	3	0.808697	0.027986
20.000	rbf	scale	4	0.808697	0.027986
20.000	rbf	auto	2	0.808697	0.027986
20.000	rbf	auto	4	0.808697	0.027986
20.000	rbf	0.1	2	0.805336	0.026994
20.000	rbf	0.1	3	0.805336	0.026994
20.000	rbf	0.1	4	0.805336	0.026994
10.000	poly	auto	2	0.803599	0.036248
10.000	poly	scale	2	0.803599	0.036248
0.010	poly	1	3	0.801961	0.028725
10.000	poly	0.1	3	0.801961	0.028725
0.050	poly	0.5	3	0.801947	0.023372
0.005	poly	5	2	0.801933	0.033381
0.001	poly	0.5	4	0.800308	0.027994
0.005	poly	1	3	0.800266	0.026129
20.000	poly	auto	3	0.800266	0.025025
20.000	poly	scale	3	0.800266	0.025025
0.050	poly	1	2	0.798571	0.033575
0.100	poly	0.5	3	0.796947	0.034777
10.000	rbf	0.5	2	0.796933	0.030977
20.000	rbf	0.5	4	0.796933	0.030977
10.000	rbf	0.5	4	0.796933	0.030977
20.000	rbf	0.5	3	0.796933	0.030977
20.000	rbf	0.5	2	0.796933	0.030977
10.000	rbf	0.5	3	0.796933	0.030977
0.050	poly	5	2	0.796933	0.036811
0.100	poly	5	2	0.796919	0.039095
10.000	poly	0.5	2	0.796919	0.039095
0.010	poly	0.1	3	0.795308	0.020919
1.000	poly	0.1	4	0.795280	0.030010
20.000	poly	0.5	2	0.793585	0.036471
10.000	poly	0.1	2	0.793571	0.030643
0.100	poly	1	2	0.793571	0.030643
20.000	poly	0.1	2	0.791905	0.040450
20.000	poly	scale	2	0.791891	0.032362
20.000	poly	auto	2	0.791891	0.032362
10.000	poly	1	2	0.791877	0.039491
10.000	poly	auto	4	0.790238	0.026784
10.000	poly	scale	4	0.790238	0.026784
0.001	poly	5	2	0.790182	0.035238
0.100	poly	0.5	2	0.790182	0.035238

20.000	poly	0.1	3	0.788557	0.023066
1.000	poly	0.1	2	0.786849	0.036584
0.010	poly	1	2	0.786849	0.036584
0.050	poly	0.5	2	0.786849	0.027299
1.000	poly	0.5	2	0.786835	0.044628
0.010	poly	5	2	0.786835	0.044628
1.000	poly	1	2	0.785196	0.034078
0.050	poly	1	3	0.783529	0.021768
0.010	poly	0.5	2	0.781863	0.018583
0.005	poly	0.5	2	0.781849	0.025718
1.000	poly	5	2	0.780140	0.039122
20.000	poly	1	2	0.780140	0.036118
0.001	poly	1	2	0.778501	0.037547
0.100	poly	0.1	2	0.778501	0.037547
0.005	poly	1	2	0.778473	0.033304
0.005	poly	0.5	4	0.776807	0.027171
20.000	poly	5	2	0.773417	0.046312
0.001	poly	5	3	0.771863	0.035828
1.000	poly	0.5	3	0.771863	0.035828
1.000	poly	auto	2	0.770084	0.029253
1.000	poly	scale	2	0.770084	0.029253
20.000	poly	scale	4	0.770070	0.037377
20.000	poly	auto	4	0.770070	0.037377
0.100	poly	1	3	0.768487	0.033624
10.000	poly	5	2	0.768347	0.049538
10.000	poly	0.5	3	0.766793	0.026645
20.000	poly	0.5	3	0.766793	0.026645
0.005	poly	5	3	0.766793	0.026645
0.050	poly	5	3	0.766793	0.026645
10.000	poly	1	3	0.766793	0.026645
10.000	poly	5	3	0.766793	0.026645
20.000	poly	1	3	0.766793	0.026645
0.010	poly	5	3	0.766793	0.026645
1.000	poly	5	3	0.766793	0.026645
20.000	poly	5	3	0.766793	0.026645
0.100	poly	5	3	0.766793	0.026645
1.000	poly	1	3	0.766793	0.026645
0.050	poly	0.1	4	0.766737	0.037581
0.010	poly	0.5	4	0.758319	0.038095
0.005	poly	1	4	0.756681	0.040986
0.050	poly	0.5	4	0.753291	0.042363
0.100	poly	0.5	4	0.749972	0.038267
10.000	poly	0.1	4	0.749930	0.039206
0.001	poly	1	4	0.749930	0.039206
20.000	poly	0.1	4	0.748277	0.030851
10.000	poly	1	4	0.746611	0.035228
20.000	poly	1	4	0.746611	0.035228
1.000	poly	0.5	4	0.746611	0.035228

20.000	poly	0.5	4	0.746611	0.035228
10.000	poly	5	4	0.746611	0.035228
10.000	poly	0.5	4	0.746611	0.035228
1.000	poly	1	4	0.746611	0.035228
1.000	poly	5	4	0.746611	0.035228
20.000	poly	5	4	0.746611	0.035228
0.001	poly	5	4	0.746611	0.035228
0.100	poly	5	4	0.746611	0.035228
0.010	poly	5	4	0.746611	0.035228
0.050	poly	5	4	0.746611	0.035228
0.100	poly	1	4	0.746611	0.035228
0.050	poly	1	4	0.746611	0.035228
0.010	poly	1	4	0.746611	0.035228
0.005	poly	5	4	0.746611	0.035228
1.000	rbf	1	3	0.738179	0.042032
1.000	rbf	1	4	0.738179	0.042032
1.000	rbf	1	2	0.738179	0.042032
20.000	rbf	1	3	0.736499	0.040051
10.000	rbf	1	2	0.736499	0.040051
10.000	rbf	1	4	0.736499	0.040051
20.000	rbf	1	2	0.736499	0.040051
10.000	rbf	1	3	0.736499	0.040051
20.000	rbf	1	4	0.736499	0.040051
0.050	poly	0.1	2	0.677871	0.038733
0.100	poly	scale	2	0.650994	0.037525
0.100	poly	auto	2	0.650994	0.037525
0.100	poly	scale	4	0.602339	0.029022
0.100	poly	auto	4	0.602339	0.029022
0.005	poly	0.1	3	0.592255	0.009637
0.050	poly	scale	4	0.563739	0.015493
0.050	poly	auto	4	0.563739	0.015493
0.010	poly	0.1	4	0.562059	0.014044
0.001	poly	0.5	2	0.562031	0.020428
10.000	rbf	5	4	0.557031	0.016656
10.000	rbf	5	3	0.557031	0.016656
20.000	rbf	5	2	0.557031	0.016656
20.000	rbf	5	3	0.557031	0.016656
20.000	rbf	5	4	0.557031	0.016656
10.000	rbf	5	2	0.557031	0.016656
1.000	rbf	5	2	0.552017	0.014468
1.000	rbf	5	4	0.552017	0.014468
1.000	rbf	5	3	0.552017	0.014468
0.100	rbf	0.5	4	0.543627	0.006309
0.100	rbf	0.5	3	0.543627	0.006309
0.100	rbf	0.5	2	0.543627	0.006309
0.050	poly	auto	2	0.541975	0.020273
0.050	poly	scale	2	0.541975	0.020273
0.005	poly	0.1	4	0.535252	0.010536

0.010	poly	auto	3	0.528529	0.009373
0.010	poly	scale	3	0.528529	0.009373
0.010	rbf	0.1	2	0.523487	0.003341
0.005	rbf	1	4	0.523487	0.003341
0.005	rbf	5	4	0.523487	0.003341
0.010	rbf	scale	2	0.523487	0.003341
0.010	poly	scale	2	0.523487	0.003341
0.010	rbf	auto	2	0.523487	0.003341
0.010	poly	auto	2	0.523487	0.003341
0.010	poly	0.1	2	0.523487	0.003341
0.005	rbf	0.1	2	0.523487	0.003341
0.010	rbf	0.5	2	0.523487	0.003341
0.001	poly	auto	2	0.523487	0.003341
0.010	rbf	1	2	0.523487	0.003341
0.010	rbf	5	2	0.523487	0.003341
0.001	rbf	0.1	2	0.523487	0.003341
0.010	rbf	scale	3	0.523487	0.003341
0.010	rbf	auto	3	0.523487	0.003341
0.010	rbf	0.1	3	0.523487	0.003341
0.005	rbf	0.5	4	0.523487	0.003341
0.005	poly	auto	2	0.523487	0.003341
0.005	rbf	0.1	4	0.523487	0.003341
0.005	rbf	0.1	3	0.523487	0.003341
0.001	poly	scale	2	0.523487	0.003341
0.005	poly	0.1	2	0.523487	0.003341
0.005	rbf	0.5	2	0.523487	0.003341
0.005	rbf	1	2	0.523487	0.003341
0.005	rbf	5	2	0.523487	0.003341
0.005	rbf	scale	3	0.523487	0.003341
0.005	poly	scale	3	0.523487	0.003341
0.005	rbf	auto	3	0.523487	0.003341
0.005	poly	auto	3	0.523487	0.003341
0.100	rbf	5	4	0.523487	0.003341
0.010	rbf	1	3	0.523487	0.003341
0.005	rbf	0.5	3	0.523487	0.003341
0.005	rbf	1	3	0.523487	0.003341
0.005	rbf	5	3	0.523487	0.003341
0.005	rbf	scale	4	0.523487	0.003341
0.100	rbf	1	4	0.523487	0.003341
0.005	poly	scale	4	0.523487	0.003341
0.005	rbf	auto	4	0.523487	0.003341
0.001	rbf	auto	2	0.523487	0.003341
0.005	poly	auto	4	0.523487	0.003341
0.010	rbf	0.5	3	0.523487	0.003341
0.001	rbf	5	3	0.523487	0.003341
0.010	rbf	5	3	0.523487	0.003341
0.001	rbf	scale	2	0.523487	0.003341
0.001	rbf	1	4	0.523487	0.003341

0.001	poly	scale	3	0.523487	0.003341
0.001	rbf	auto	3	0.523487	0.003341
0.001	rbf	0.5	4	0.523487	0.003341
0.050	rbf	0.5	4	0.523487	0.003341
0.050	rbf	1	4	0.523487	0.003341
0.001	poly	auto	3	0.523487	0.003341
0.050	rbf	5	4	0.523487	0.003341
0.001	rbf	0.1	3	0.523487	0.003341
0.001	poly	0.1	4	0.523487	0.003341
0.050	rbf	5	3	0.523487	0.003341
0.001	poly	0.1	3	0.523487	0.003341
0.001	rbf	0.1	4	0.523487	0.003341
0.001	poly	auto	4	0.523487	0.003341
0.001	rbf	0.5	3	0.523487	0.003341
0.001	rbf	1	3	0.523487	0.003341
0.001	rbf	auto	4	0.523487	0.003341
0.100	rbf	5	2	0.523487	0.003341
0.001	poly	scale	4	0.523487	0.003341
0.001	rbf	scale	4	0.523487	0.003341
0.001	rbf	scale	3	0.523487	0.003341
0.050	rbf	1	3	0.523487	0.003341
0.100	rbf	1	2	0.523487	0.003341
0.100	rbf	5	3	0.523487	0.003341
0.010	poly	scale	4	0.523487	0.003341
0.010	rbf	auto	4	0.523487	0.003341
0.010	poly	auto	4	0.523487	0.003341
0.010	rbf	0.1	4	0.523487	0.003341
0.001	poly	0.1	2	0.523487	0.003341
0.001	rbf	0.5	2	0.523487	0.003341
0.010	rbf	0.5	4	0.523487	0.003341
0.010	rbf	1	4	0.523487	0.003341
0.001	rbf	1	2	0.523487	0.003341
0.010	rbf	5	4	0.523487	0.003341
0.050	rbf	0.5	3	0.523487	0.003341
0.001	rbf	5	2	0.523487	0.003341
0.005	poly	scale	2	0.523487	0.003341
0.005	rbf	scale	2	0.523487	0.003341
0.100	rbf	1	3	0.523487	0.003341
0.050	rbf	0.5	2	0.523487	0.003341
0.050	rbf	1	2	0.523487	0.003341
0.050	rbf	5	2	0.523487	0.003341
0.005	rbf	auto	2	0.523487	0.003341
0.001	rbf	5	4	0.523487	0.003341
0.010	rbf	scale	4	0.523487	0.003341

=====

=====

```
=====
SUMMARY BY KERNEL TYPE
=====
```

LINEAR Kernel:

```
Best score: 0.8574
Worst score: 0.8440
Average score: 0.8540
Number of combinations: 144
Best combination: C=0.001, Gamma=scale, Degree=4
```

RBF Kernel:

```
Best score: 0.8674
Worst score: 0.5235
Average score: 0.6560
Number of combinations: 144
Best combination: C=0.05, Gamma=auto, Degree=2
```

POLY Kernel:

```
Best score: 0.8590
Worst score: 0.5235
Average score: 0.7264
Number of combinations: 144
Best combination: C=0.1, Gamma=auto, Degree=3
```

```
[34]: # Use the best model from hyperparameter tuning
svm_model = best_svm

print("Best SVM model (from hyperparameter tuning) is ready!")
print(
    f"Number of support vectors: {len(svm_model.named_steps['svm'].
    ↪support_vectors_)}"
)
print(f"Support vectors per class: {svm_model.named_steps['svm'].n_support_}")
```

```
Best SVM model (from hyperparameter tuning) is ready!
Number of support vectors: 437
Support vectors per class: [221 216]
```

```
[35]: # Make predictions using the pipeline (it will handle standardization
    ↪automatically)
y_train_pred = svm_model.predict(X_train)
y_test_pred = svm_model.predict(X_test)
```



```

# Calculate accuracies
train_accuracy = accuracy_score(y_train, y_train_pred)
test_accuracy = accuracy_score(y_test, y_test_pred)

print("=" * 50)
print("SVM MODEL PERFORMANCE")
print("=" * 50)
print(f"\nTrain Accuracy: {train_accuracy:.4f} ({train_accuracy*100:.2f}%)")
print(f"Test Accuracy: {test_accuracy:.4f} ({test_accuracy*100:.2f}%)")
print("\n" + "=" * 50)
print("\nConfusion Matrix (Test Set):")
print(confusion_matrix(y_test, y_test_pred))
print("\nClassification Report (Test Set):")
print(
    classification_report(
        y_test, y_test_pred, target_names=["0: No Heart Disease", "1: Heart_
↪Disease"]
    )
)

```

```

=====
SVM MODEL PERFORMANCE
=====

```

```

Train Accuracy: 0.8758 (87.58%)
Test Accuracy: 0.8533 (85.33%)

```

```

=====

```

```

Confusion Matrix (Test Set):
[[67 11]
 [11 61]]

```

```

Classification Report (Test Set):

```

	precision	recall	f1-score	support
0: No Heart Disease	0.86	0.86	0.86	78
1: Heart Disease	0.85	0.85	0.85	72
accuracy			0.85	150
macro avg	0.85	0.85	0.85	150
weighted avg	0.85	0.85	0.85	150

```

[36]: # Plot ROC Curve
print("=" * 70)

```

```

print("ROC CURVE ANALYSIS")
print("=" * 70)

# Get decision function scores (SVM doesn't output probabilities by default)
# We need to use decision_function() to get scores for ROC curve
y_train_scores = svm_model.decision_function(X_train)
y_test_scores = svm_model.decision_function(X_test)

# Calculate ROC curve for test set
fpr, tpr, thresholds = roc_curve(y_test, y_test_scores)
roc_auc = roc_auc_score(y_test, y_test_scores)

# Also calculate for training set for comparison
fpr_train, tpr_train, thresholds_train = roc_curve(y_train, y_train_scores)
roc_auc_train = roc_auc_score(y_train, y_train_scores)

# Plot ROC curves
plt.figure(figsize=(10, 8))
plt.plot(
    fpr, tpr, color="darkorange", lw=2, label=f"Test ROC curve (AUC = {roc_auc:.4f})"
)
plt.plot(
    fpr_train,
    tpr_train,
    color="blue",
    lw=2,
    linestyle="--",
    label=f"Train ROC curve (AUC = {roc_auc_train:.4f})",
)
plt.plot(
    [0, 1],
    [0, 1],
    color="navy",
    lw=2,
    linestyle="--",
    label="Random Classifier (AUC = 0.5000)",
)
plt.xlim([0.0, 1.0])
plt.ylim([0.0, 1.05])
plt.xlabel("False Positive Rate (1 - Specificity)", fontsize=12)
plt.ylabel("True Positive Rate (Sensitivity)", fontsize=12)
plt.title("ROC Curve - SVM Model", fontsize=14, fontweight="bold")
plt.legend(loc="lower right", fontsize=11)
plt.grid(alpha=0.3)
plt.tight_layout()
plt.show()

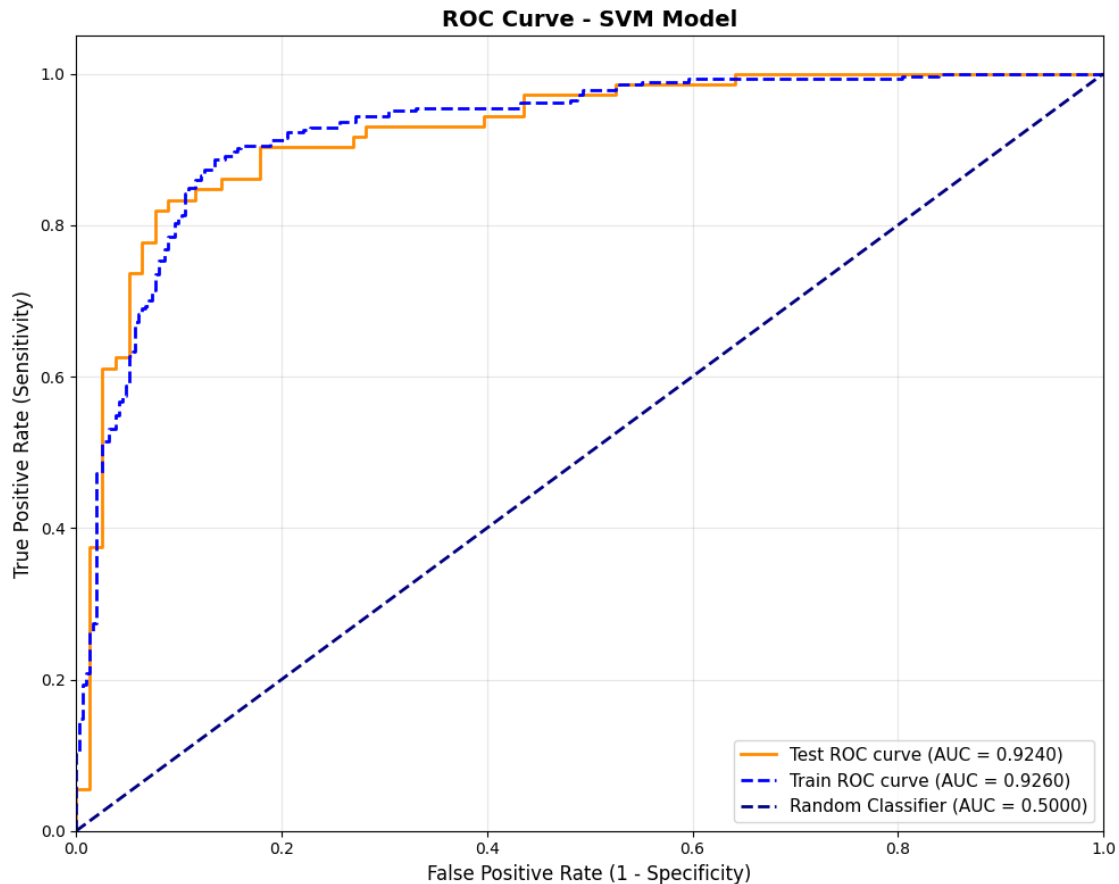
```

```

print(f"\nROC-AUC Score (Test Set): {roc_auc:.4f}")
print(f"ROC-AUC Score (Train Set): {roc_auc_train:.4f}")
print(f"\nInterpretation:")
print(f" - AUC = 0.5: Random classifier (no discriminative ability)")
print(f" - AUC = 1.0: Perfect classifier")
print(f" - AUC > 0.8: Good classifier")
print(f" - AUC > 0.9: Excellent classifier")
print(
    f"\nOur model has an AUC of {roc_auc:.4f}, which indicates {'excellent' if
    ↪roc_auc > 0.9 else 'good' if roc_auc > 0.8 else 'moderate'} performance."
)
print("=" * 70)

```

ROC CURVE ANALYSIS



ROC-AUC Score (Test Set): 0.9240

ROC-AUC Score (Train Set): 0.9260

Interpretation:

- AUC = 0.5: Random classifier (no discriminative ability)
- AUC = 1.0: Perfect classifier
- AUC > 0.8: Good classifier
- AUC > 0.9: Excellent classifier

Our model has an AUC of 0.9240, which indicates excellent performance.

=====

```
[37]: # Plot Confusion Matrix
cm = confusion_matrix(y_test, y_test_pred)

group_names = ["True Negative", "False Positive", "False Negative", "True_
↵Positive"]
group_counts = ["{0:0.0f}".format(value) for value in cm.flatten()]
labels = [f"{v1}\n{v2}" for v1, v2 in zip(group_names, group_counts)]
labels = np.asarray(labels).reshape(2, 2)

cm_df = pd.DataFrame(
    cm,
    index=["Actual Negative", "Actual Positive"],
    columns=["Predicted Negative", "Predicted Positive"],
)

plt.figure(figsize=(6, 5))
sns.heatmap(cm_df, annot=labels, fmt="", cmap="Blues", cbar=False)

plt.title("Confusion Matrix - Test Set (SVM)")
plt.ylabel("True Label")
plt.xlabel("Predicted Label")
plt.tight_layout()
plt.show()
```

